kayfabe

A Mode-2 NVIDIA GPU forwarding stack for KVM/QEMU guests

An architecture whitepaper, written to be attacked

Subject. Two repositories on one machine. `/workspace/nvidia-gpu-passthrough` — the C research artifact, 753 commits, 44 783 lines of C, developed 2026-05-24 to 2026-08-01; it works on real hardware, and it is no longer history: it is maintained as this project's *standing differential oracle*. `kayfabe` — a clean-slate Rust rewrite, 414 commits, 120 979 lines of implementation (plus a 3 284-line detached ABI generator), 118 160 lines of test code, and 2 676 lines of C QOM shim, developed 2026-07-24 to 2026-08-03. This document is pinned to Rust HEAD `f33e1c8` and C HEAD `8001cb5`.

★★★ What moved since the previous pin (`15651b1`) is the sentence this paper has led with three times. It read: *"No part of the Rust codebase has ever driven a real GPU."* **That is now false, and retiring it is the single most important edit in this revision.** Two things happened on real hardware, and the discipline that matters is that *they are two different experiments and they have not met*:

(1) A stock, unpatched NVIDIA 580.159.04 open kernel module, in a KVM guest, binds to our emulated GA106. [measured] 2026-08-01, rev `55a106f`, and on every boot since: the device presents `10de:2504`, the guest loads five modules (`nvidia`, `nvidia_modeset`, `nvidia_drm`, `video`, `ecc`), and `nvidia-drm` initialises on `0000:00:03.0`. ⊘ **And the boot does not complete.** `RmInitAdapter` fails, no `/dev/nvidia*` node is created, and `nvidia-smi` finds no device. There is no `cuInit`, no `cuCtxCreate` and no compute in Mode 2 anywhere in this codebase.

(2) A guest-shaped copy-engine request was executed by a real host GPU and read back. [measured] 2026-08-03 on vast `46529600` (RTX 3060 / GA106, host driver 580.159.04 open), twice at two revisions (`147c069` and `5511cda`) with byte-identical evidence: `CeEvidence::copied() == true` — a real GA10x GPFIFO entry naming five real CE method runs, carried by `read_pushbuffer` → `decode_run` → `partition_ce` → `plan_ce` → `Worker::execute` through production crates, 4 096 bytes moved, the release semaphore landed, read back through an *independent* second mapping. The predicate was watched to **fail** first on the same hardware under a live mutation. ⊘ **No live guest drove it**, and the test says so in its own header: the entry is a guest's *bytes* and a guest's *declarations*, not a booted guest's submission. The boot in (1) dies inside `RmInitAdapter`, long before any channel is scheduled, so it never rings anything for (2) to carry.

★★★ Second, and it is why (1) and (2) have not met: the execution plane is the wall, and it cannot be faked. The boot dies in the memory manager's scrubber — as of 2026-08-03 at `mem_utils.c:2022`, registering the scrubber channel's completion notifier, one step past the channel schedule that held it before. [inferred] from source and immediately past both, `memmgrInitCeUtils_IMPL` calls `memmgrTestCeUtils` **unconditionally**: it writes a sentinel to framebuffer, issues a copy with `TRANSFER_FLAGS_PREFER_CE`, reads it back and asserts equality. **There is no control-shaped answer to a read-back test**, and its status returns into an `NV_ASSERT(0)` that is fatal for any non-`NV_OK` in this phase. Every escape hatch is closed by citation — most sharply `!IS_SILICON`, whose three flags are *declared in the driver's headers and assigned nowhere in the tree*, so it is structurally constant true: *this is not a lie we decline to tell, it is a lie we cannot tell.* ⊘ **No boot has ever reached** `memmgrTestCeUtils` — the last one died two statements short of it — so that specific wall is [inferred], and this paper marks it so. Increments E0–E7 exist to move the driver of (2)'s predicate from a ladder to a guest; §8.2 is that story, including the walls it hit and the one it has not.

★★ Third, and it bounds the oracle in a way the previous revision did not know: there is a FIFTH limit, and on it the C is POSITIVELY WRONG rather than blind. The four measured limits of a C recording still stand (§3.1) and are still the right frame. But the C's captured control table `mode2_initctrl_ga106.h` has 56 rows of which **11 (19.6%) carry** `dlen = 0` — the reply body was never captured — and asked of a real GA106, **nine of the eleven are contradicted**. `CE_GET_FAULT_METHOD_BUFFER_SIZE` decodes from its empty row as size 0; the part answers **20 480**, and RM DMAs fault records into a buffer of exactly that size. ⊘ **An empty capture is evidence of nothing, not evidence of emptiness.** ★ And note how it survived four rungs: a gate demanding a `C:` citation was *satisfied* by a row citing the empty body as corroboration. **A citation gate checks that a claim is sourced; it can never check that the source says what the claim says.**

Fourth: for the first time this project owns a trace of a boot that SUCCEEDS. A recorder inside CPU-RM (two call sites, complete elements, both directions) captured a real GA106 booting to a working `nvidia-smi`: 1 076 records, 104 distinct controls, zero dropped, zero replies declaring params with no bytes present. From it the ladder is *enumerated* rather than discovered one boot at a time — 53 controls beyond what the failing `cap1` capture reaches — and a real GSP is seen to **refuse 13 controls on the boot that works**, with CPU-RM's own source carrying an explicit provision for 7 of the 7 it can name. **Refusal is ordinary protocol behaviour, not only our posture.** ⚠ Those artifacts live on branch `replay-conformance`, not on `master`, and §8.3 says so where it uses them.

Stance. Every number here was measured from the tree while writing, not copied from a document and not inherited from an adjacent sentence — every error previously found in this paper came from restating its own neighbouring prose. **Two numbers supplied to this revision in its own brief were checked and found wrong**, and the measured values are what appear below (§10 rows 31–32). Where a project document and the code disagree, the code wins

and the disagreement is recorded. Roughly a third of the document is about what does not work, is not built, or is not known.

Contents

1 What this document is, and how to attack it

This is a review document. Its job is to describe kayfabe accurately enough that a competent systems engineer who has never seen it can (a) explain what it does and why it is hard, (b) find any component in the tree, (c) judge whether the claims are believable, and (d) find the weak points without help. It is not a status report and it is not a pitch.

Four consequences shape everything below.

Numbers trace to something. Every count, score and measurement names its source. Where a number is an estimate it says so; where a claim could not be verified it is marked [unverified] rather than softened. Appendix A gives the exact commands used.

Unbuilt is stated as unbuilt. Most of the planned system does not exist. The build order is deliberate, but a document that blurs "designed" and "running" is worthless for the stated purpose. The layer diagram (Figure 1) and the data-path diagram (Figure 3) both carry explicit **NOT BUILT** bands.

The uncomfortable section is the longest. §8 is where to start if you only read one section.

This document applies the project's own doctrine to itself. kayfabe's testing doctrine opens with the rule *"a green instrument on an unexercised path is worse than no instrument — it reads as evidence, and nobody re-checks a zero"* (`docs/design/testing_doctrine.md` §1). The same rule applies to a whitepaper. So where this document says a thing is tested, it also says what that test would have to be wrong about for the claim to fail.

The single most important paragraph in this paper, and it is no longer one sentence. The previous three revisions led with *"no part of the Rust codebase has ever driven a real GPU."* **That is retired.** What replaces it is longer because the truth is now a *conjunction*, and every reader who takes only half of it will be wrong in one direction or the other.

What has happened. A stock, unpatched NVIDIA driver in a KVM guest binds to our emulated GPU and loads its whole module stack. Real host RM ioctls are issued by a real, sandboxed, unprivileged isolate process spawned in response to the guest's own action. A real host copy engine has moved 4 096 bytes because our production core decoded a GA10x pushbuffer, and the bytes were read back through an independent mapping. Three binaries exist (`kayfabe-isolate`, `kayfabe-rm-ladder`, `kayfabe-sandbox-probe`), plus a 2 676-line C QOM shim that links our `staticlib` into a real QEMU. **None of that was true at the previous pin.**

What has not happened, and it is the larger half. The boot does not complete. `RmInitAdapter` fails, every time, on every box. No `/dev/nvidia*` node is created in the guest, `nvidia-smi` finds no device, and **there is no** `cuInit`**, no** `cuCtxCreate`**, and no compute of any kind in Mode 2** — a grep for the CUDA entry points across every crate and test returns doc comments about the C artifact and zero call sites. **And the two hardware results above are two different experiments that have not met**: the copy-engine acceptance is driven by a test harness feeding a guest-shaped GPFIFO entry through the production path, because the booted guest dies long before it would ring a doorbell. On every boot to date the device reports `doorbells: 0 arrived`.

★ **The** `cuCtxCreate` $\rightarrow$ **2048² matmul at** `bad=0 maxerr=0` **belongs to the C artifact, not to this codebase**, and this paper names the subject every time it quotes that result. Everything below describes a system that is real, tested, internally coherent, and that has now met a small and carefully-delimited part of the thing it exists to talk to.

2 The idea

2.1 The problem

Giving a virtual machine a real GPU normally means one of two things. **IOMMU passthrough** hands the whole physical device to exactly one guest: simple, fast, and it gives every other guest nothing. **Vendor virtualization** (SR-IOV, NVIDIA vGPU) shares the device properly, but it exists on datacenter parts under licences, not on the consumer cards most people own.

kayfabe is a third option: **share one commodity GPU among several untrusted virtual machines, with per-guest-process blast-radius containment, using only unprivileged host processes**. The multi-tenant part is the point. A single cross-tenant leak, or a hostile guest that can wedge the box, would be fatal to the thesis.

2.2 Vocabulary

The rest of this paper leans on five terms.

- **RM** — NVIDIA's *Resource Manager*, the object model inside the driver. Everything a GPU program touches is an RM object with a handle: clients, devices, address spaces, channels, memory, engine contexts. Guest software allocates and frees them through ioctls and RPCs.

- **GSP** — the *GPU System Processor*: on modern NVIDIA hardware a real CPU core on the GPU die, running a firmware copy of much of the resource manager. The host driver no longer programs most of the hardware directly; it sends RPC messages to the GSP over a ring buffer in memory.
- **VAS / PDB** — a GPU virtual address space, and its *page directory base*: the physical address of the root of that address space's page tables. The GPU's MMU keys page tables by PDB, which makes **the PDB the real memory boundary** — not the client handle.
- **Channel / vChid** — channels are the GPU's work-submission queues; the *virtual channel id* is how a submission is routed to one. **vChid is the execution boundary.**
- **Doorbell** — the MMIO register write that says "channel N has work waiting". The value written is a token that encodes the vChid.

2.3 Mode 1 and Mode 2

The predecessor project defines two modes, and kayfabe is only the second.

Mode 1 forwards the guest's *userspace* NVIDIA ioctls to a host stub that replays them against the real `/dev/nvidia*`. It needs a guest kernel module and a virtio transport, so the guest is modified, and it caps you at "a Linux guest running our module". It is mature: 22 real GPU applications at host parity, Vulkan, OpenGL, a working desktop present path, five security audits.

Mode 2 inverts it. The guest runs the **stock, unmodified NVIDIA kernel driver** against an *emulated* PCI device that presents a plausible GPU and a *faked GSP*. We implement the RPC ring the driver talks to and answer as the firmware would. The guest driver believes it owns hardware: it allocates RM objects, builds GPU page tables, creates channels, and rings doorbells. We are on the other side of that conversation, and from the protocol traffic we reconstruct what the guest program was actually trying to do — then do the equivalent thing on a real host GPU through ordinary unprivileged userspace operations, inside a sandbox.

Mode 2 needs *nothing* from the guest, which is the entire reason it is worth the difficulty.

2.4 The correctness criterion, and why it is not "replay everything"

The naive reading of Mode 2 is "intercept the guest's privileged operations and perform them on the host". That is wrong, and the project's forwarding model says so explicitly:

> *"The job of Mode-2 is **not** to replay those low-level steps on the host. It is to **recover the original userspace-level intent and re-express it as the normal, unprivileged host userspace operations** — exactly the operations Mode-1 forwards directly."*
> [C: docs/design/mode2_forwarding_model.md]

The guest's kernel RM has *already* decomposed an unprivileged userspace intent (`cuCtxCreate`, `cuMemAlloc`, a kernel launch) into privileged GSP-internal steps. Replaying those steps on the host would require privilege we deliberately do not have — and the project has a named lesson about this: an `NV_ERR_INSUFFICIENT_PERMISSIONS` from the host stub *never* means "we lack a privilege we should have", it means "we forwarded at the wrong layer". So the operational split is:

- **Case 1** — the RPC essentially *is* the userspace operation. Re-issue it roughly 1:1 on the host, through the isolate, unprivileged, and let the host's own kernel RM legitimately re-derive the privileged steps for itself.
- **Case 2** — a GSP-internal control with no userspace equivalent (`PROMOTE_CTX`, `GET_CTX_BUFFER_INFO`). **Acknowledge the guest; do nothing on the host.** The host has already done the equivalent for its own context.

The correctness criterion that licenses this is **observable end-state equivalence**, not step fidelity. It is perfectly correct for some guest-kernel operations to complete instantly as fakes, *as long as* the one operation that actually carries the work triggers the real host-side chain. The hard boundary on the other side is equally explicit: a completion that gates guest *userspace* must be a real host write. Forging one would make the whole project a lie, and the design forbids it by name.

2.5 Why this is hard

1. **The protocol is not documented and not stable.** It varies on two independent axes: the *driver version* (wire layouts) and the *GPU generation* (silicon behaviour). kayfabe separates these as "Axis A" and "Axis B" and gives each its own trait seam, because conflating them is its own bug class.
2. **Some things cannot be faked at all.** GR's golden context is produced by FECS/GPCCS microcode on real silicon. There is no way to fabricate one. The design's response is to never emulate an engine — forward the guest's own pushbuffer and let real silicon produce the context.

3. **Address translation is a trust problem, not a lookup problem.** The guest builds GPU page tables in memory it controls. Reading them at an arbitrary instant means acting on bytes an adversary can change under you.
4. **Identity collides by construction.** Guest handles and guest VAs are per-process namespaces. Two processes using identical values is normal. Anything that keys on them is wrong (Figure 4).
5. **The failure modes are global.** A mis-resolved address is not a wrong answer; it is a host GPU MMU fault that takes out a channel, and in the worst case a state the guest driver cannot recover from without a full VM restart.
6. **Everything is concurrent.** N guest vCPUs, several guest processes, several host isolate processes, an interrupt path, and a host RM that *serializes every ioctl per client* so that one wedged verb blocks every isolate in D-state.
7. **★★★ And the last one, which this project learned by hitting it: the driver does not only ask questions, it runs experiments.** A protocol can be answered. A *functional test* cannot. Partway through initialisation the guest driver writes a sentinel to framebuffer, asks a copy engine to move it, reads it back and compares — unconditionally, with no code path that skips it on the hardware we target. **There is no reply that satisfies a read-back comparison.** Everything above is about recovering intent from a conversation; this is the point at which the conversation stops and something has to actually happen. §8.2.

3 Provenance: the C artifact, and what it actually proved

kayfabe is a rewrite, not a greenfield project. The C artifact at `/workspace/nvidia-gpu-passthrough` is the working prior implementation and the differential oracle for everything the rewrite claims about NVIDIA's behaviour. Being precise about what it did and did not prove is the foundation of every other claim here.

Claim	Mode	What was actually measured
22 real GPU apps at host parity	1	**True.** `tests/perf/realapp_matrix.md`, 2026-06-01. Same binary, host and guest, strictly serially on one RTX 3060, with a correctness check alongside throughput; parity gate ≥ 0.90. 10 CUDA benchmarks, 7 PyTorch, 2 LLM, 3 graphics. **Caveat the doc states itself:** NVENC is *partial*, not parity (`InitializeEncoder` fails); NVDEC excluded (broken on the host baseline too).
7B LLM at 63 tok/s	1	**True but mis-framed.** 63.4 tok/s is a Mode-1 A/B endpoint after a caching fix, not a headline result. The audited same-day parity figure is **host 67.8 → guest 66.0 tok/s, 0.97×** (Qwen2.5-7B Q4_K_M, `-ngl 99`, RTX 3060). Quote the ratio.
Mode-2 bare-metal parity	2	**True, one workload.** 49.9 tok/s Mode-2 vs 47.5 tok/s host-native *on the same RTX 3050*, i.e. zero forwarding overhead within noise. The entire earlier 20→50 tok/s gap was **nested-virt vmexit tax**, not Mode-2 design — the vast.ai bench is itself a VM, so the Mode-2 guest was L2. The project retracted its own earlier "~21% residual" claim as an apples-to-oranges comparison across two different GPUs.
Mode-2 CUDA correctness	2	**True.** Full fake-boot + GSP RPC + GMMU + CE + IRQ; `cuInit`, `cuCtxCreate`, byte-exact matmul; PyTorch 2.5.1 *single-process*. Bugs #12 (second-context hang) and #13 (multi-iteration hang) resolved on hardware.
What the C artifact never proved		
Mode-2 multi-process	2	**Never.** Bug #14 is open. Root-caused on hardware 2026-07-24 by a host Xid 31 `FAULT_PDE` naming the loser's exact GR channel: the second process's identical guest VAs were never published into *its own* host address space.
Mode-2 sequential processes	2	**Broken at HEAD.** Measured 2026-07-25: exactly *one* CUDA process works per QEMU lifetime; the second fails `cuInit`→999 regardless of how the first exited. The "fresh boot per clean run" bench discipline *is* this bug, not a precaution.
Mode-2 graphics / display	2	**Never.** All Vulkan/OpenGL/present results are Mode 1.
Mode-2 on the 22-app matrix	2	**Never run.** It is Mode-2's definition of done.
Multi-GPU, MIG, non-Ampere	—	**Never.** Ampere GA106 only. Turing/Ada/Hopper/Blackwell rows in the ABI doc are all marked *assumption — verify*.
Mode-2 security posture	2	**Not audited.** The five security audits are Mode-1-era. The 2026-07-25 bench found live guest-reachable Mode-2 defects, including parsing arbitrary guest RAM as GSP RPC elements after a guest-triggered driver reload.

The structural reason for the rewrite. #14 is not a bug that could be patched. The architecture document states it in one line: *"the emulator's completion, execution, isolate, and GPA-arena planes are each a single-process singleton, and they fail in dependency order."* Four rounds of investigation each found a different plane; they were one structural fact observed four times. Making per-process separation structural rather than retrofitted is the rewrite's founding requirement, and it is why the diagram in Figure 4 is the centre of the design.

One provenance correction worth carrying. Two C design documents state the C baseline encodes *"~14 months of quirk capture"*. Git says the first commit is 2026-05-24 and the last is 2026-08-01: **ten weeks, 753 commits**. Either the figure means agent-hours, or it is wrong. Use the verifiable number.

3.1 ★★ The C artifact is a standing oracle — and it has FIVE measured limits, the fifth different in kind

Two virtualizers facing the same guest driver see identical MMIO iff they behave identically, so a divergence at guest access N localises the fault to our reply at $N-1$: one message, not one layer. That is why the C was promoted from history to a *standing differential oracle*, and why it is still maintained. Five committed captures

exist (`traces/mode2_c_reference/`), all dense with `n_errors=0`; two of them are committed *decompressed* into this repository so the harness needs no decoder and no third-party crate.

Capture	Records	Hermetic	What it is for
`cap1_coldboot_hermetic`	359 062	yes	cold GSP bring-up to `GSP_INIT_DONE`; 139 821 PROM/VBIOS reads
`cap1b_..._d6`	360 725	yes	★ the same experiment re-taken with the continuation elements *witnessed* — **the only trace a replay can be closed over**
`cap2_stalequeue_negative`	886 999	no	the `WPR2-already-up` chain
`cap2b_stalequeue_nofn47`	862 940	no	★ the real negative: **378 GSP elements parsed out of arbitrary guest RAM and answered** `NV_OK`
`cap3_matmul_forwarding`	532 824	no	the decision planes; `cuCtxCreate` → matmul at `bad=0 maxerr=0`

The four limits of a C *recording*, all pre-registered before the capture. (**i**) The completion plane has *no* C oracle at all — the C never observes a host completion source, it *forges* completions, and a grep over its isolate verbs finds seventeen call sites and zero poll/event verbs. **A green diff says nothing about it**, and that is the likeliest source of false confidence. (**ii**) The diff will never be green end-to-end, and a green diff would itself be the bug: the C has no refusal vocabulary, so every must-differ row is a position where it emits a positive event and we emit a refusal. (**iii**) Any forwarding-mode trace is non-hermetic by construction — `pci_dma_map` is an uninstrumented channel and the host GPU DMAs into guest RAM behind every recorder. (**iv**) The archived logs are unusable: the C's sequence counter is incremented *inside* its log statement, so it counts logged BAR0 accesses only. ★ The load-bearing good news is that a single-counter total order is nonetheless *sound* — the device creates no thread, bottom half or timer, and all four `MemoryRegionOps` run under the BQL — which upgrades a previously [inferred] assumption to an evidenced one.

★★★ **The FIFTH limit, and it is different in kind: here the oracle is POSITIVELY WRONG, not blind.** The C's captured control table `mode2_initctrl_ga106.h` has **56 rows**. A census (`scripts/census_initctrl.py`, pinned by a test) splits them: **29 complete**, **16 truncated** ($0 <$ `dlen` $<$ `psize`) and **11 empty** (`dlen = 0`, 19.6%). All eleven empty rows were asked of a real GA106 — an RTX 3060 running the open 580.159.04 modules rebuilt with a probe, 2026-08-01, `traces/real_ga106/` — and **nine of the eleven are contradicted**.

The sharpest is `0x20802a08`, `CE_GET_FAULT_METHOD_BUFFER_SIZE`: its empty row decodes as size **0**; the part answers **20 480**. RM DMAs CE fault records into a buffer of exactly that size, so trusting the empty row was **a buffer overrun with a hardware writer**, not merely a wrong number.

★ **And the two that are *not* wrong are the important ones.** `0x20800a70` has `psize = 0`, so nothing could have failed to be captured — checkable from the row itself. `0x20800a4c` genuinely answers zero on this part because SMC is disabled — and **nothing about the row says so**. It is byte-identical to the nine that are wrong. ⊘ **An empty capture is evidence of nothing, not evidence of emptiness**: believing it would have been right once and wrong nine times *for the same reason*. The gate that now enforces this refuses the predicate `psize` $> 0 \wedge$ `dlen` $= 0$ rather than a list of eleven ids, because a list is a smaller true statement that goes stale at the 57th row.

⊘ **And the third class is a live hole this paper will not paper over.** The 16 truncated rows are *larger* than the empty class and **have not been measured**. A truncated row *passes* a `dlen == 0` test because it has a body, and its uncaptured tail arrives as zeros — the exact failure `0x20802a08` cost four rungs, with the one signal that caught it removed. All sixteen kept-prefixes end in zero bytes, so the trim hypothesis is dead and the tail is genuinely *unknown*.

★ **This scopes the oracle rather than devaluing it.** It remains the only implementation a real NVIDIA driver has ever accepted end to end, and its non-empty bodies are corroborated by hardware where they have been checked. What is demoted is one move: reading an empty `ctl_` array as *"hardware says zero"*.

★★★ **How it survived four rungs, which is the part that generalises.** A gate demanding a `C:` source citation was *satisfied* by a row that cited the empty body *as corroboration*. **Citing the oracle is not the oracle being right. A citation gate checks that a claim is sourced; it can never check that the source says what the claim says.** The sibling was found from the opposite side in the same week: three shipped rows cited `mode2_initctrl_ga106.h` lines belonging to *different* controls — every value right, every address wrong, the citation gate satisfied throughout. And one turn deeper still: the replacement gate's own first version accepted the *phrase* `"real GA106"` as evidence, which a row satisfied with a claim about hardware manufactured out of the empty capture. It now demands a path to a committed artifact. **A gate that checks for the vocabulary of evidence can be satisfied by writing the vocabulary down.**

Has any replay been closed? No — and the wall changed kind, which is the result. The `cap1` replay stops at transaction **978**, on a *missing observation*: the C never recorded the continuation elements, so the run dies at a read the capture does not contain. `cap1b` witnesses them (32/32, proven against a control not to have changed the reply stream) and the limit moves to **1028** — where every read is *observed* and the refusal is `QueueFull`, because the C's only accesses to the status-queue header are *writes*. **That is oracle blindness, not a Rust defect**: the guest really is advancing a pointer we are right to consult, and no capture taken from this C can ever contain it. ⊘ **Do not treat** `QueueFull` **as a bug to be tuned away** — relaxing our flow control to get a greener diff would delete a correctness property the oracle simply cannot see. *The diff is the instrument; the instrument does not get to edit the subject.*

And the oracle is perishable. Recording a C trace needs a running C deployment: a built QEMU, a guest pinned to 6.8.0-117, and host driver 580.159.04. That deployment has died with a rented box before and will again. **Recorded traces are the durable artifact; a bootable C on a rented box is not.**

4 The architecture

4.1 Layers and ports

The one defining constraint. The logic core is a pure state machine over guest-supplied bytes: no OS, no syscalls, no hypervisor types, no wall clock, no `#[repr(C)]` NVIDIA struct layouts, and no concrete GPU-generation or driver-version name anywhere. Everything effectful crosses a trait seam. `unsafe_code = "forbid"` is a workspace lint; every core type is compile-time-asserted `Send + Sync`.

This is enforced, not intended. Four CI greps run on every push: a *hexagonal boundary* grep that fails if `eventfd|epoll|timerfd|rawfd|libc|O_NONBLOCK` appears in any of the 11 pure crates; a *VMM-vocabulary* grep that fails if any QEMU concept (`BQL`, `MemoryRegion`, `qemu_bh`, `iothread`, ...) appears in those 11 plus `kayfabe-rt`; a *generation-name* grep over 10 of them that fails on any concrete chip or GPU-generation name outside a comment; and a *GPA-accessor* gate that asserts `.gpa_read(/.gpa_write(` appears **exactly once** in the entire forwarding crate, and **nowhere** in the other nine.

The claim that pays for this constraint is the **adapter contract**: bringing up a new GPU generation is `impl Arch for <Gen>` in an adapter crate with *zero edits to any logic crate*. The standing proof is that the whole suite runs the real core against `MockArch`, whose encodings are deliberately non-NVIDIA — if the core had absorbed a real encoding anywhere, the mock would not work.

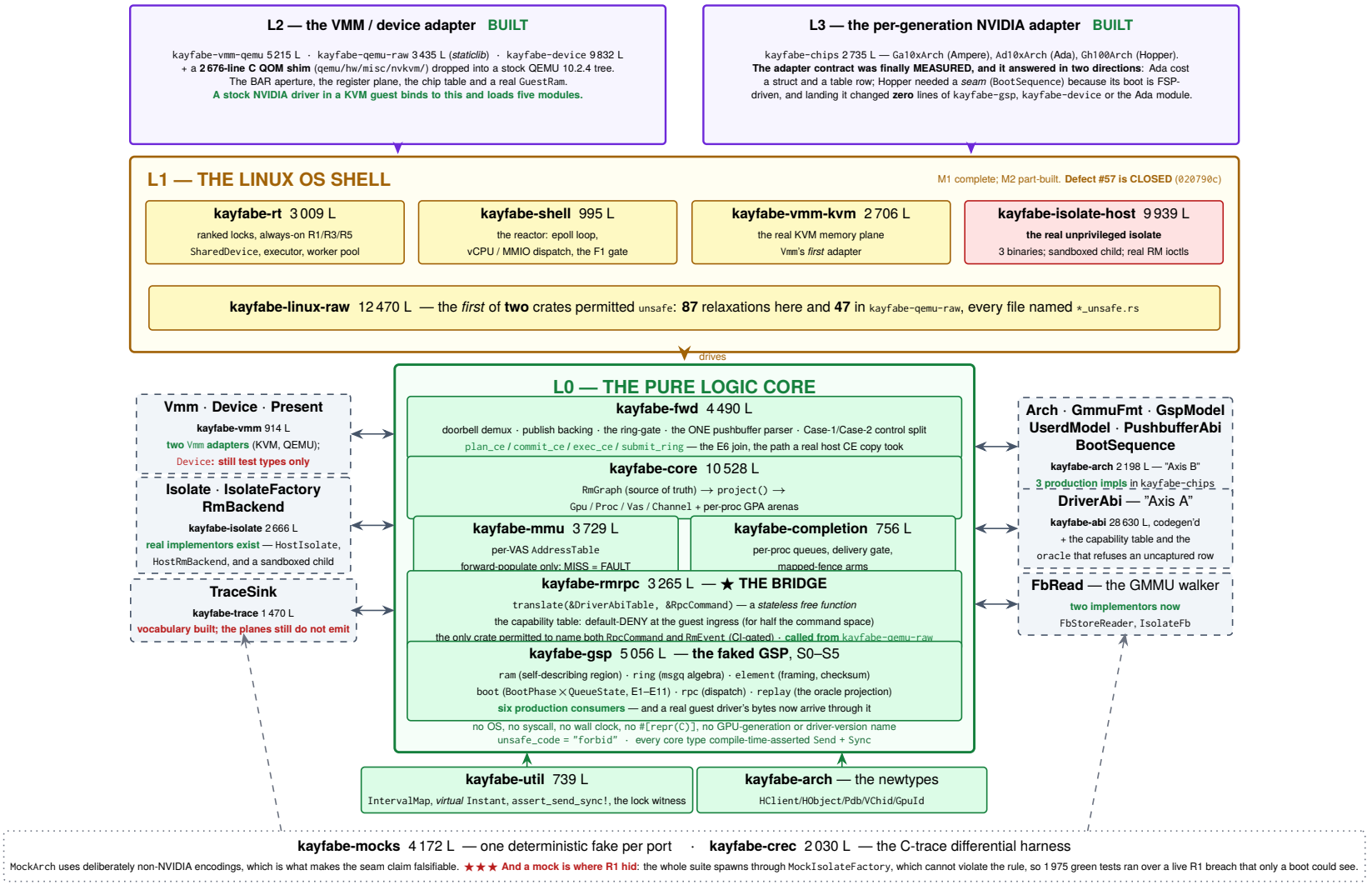


Figure 1: The layer model and the hexagonal ports. Line counts are measured at `f33e1c8` (`crates/<name>/src/**/*.rs`; `kayfabe-abi`'s figure excludes its detached offline generator, 3 284 lines). ★★★ **The bands inverted at this revision**: L2 and L3 were drawn as dashed **NOT BUILT** boxes at the previous pin and are now built, shipped and booted against. Implementor census, counted from `impl <Trait> for` across the tree and re-derived rather than carried over: `Arch` has **three** production implementations (`Ga10xArch`, `Ad10xArch`, `Gh100Arch` in `kayfabe-chips`); `Isolate`, `IsolateFactory` and `RmBackend` have real ones in `kayfabe-isolate-host`; `Vmm` has *two* real adapters (`KvmVmm`, `QemuVmm`); `FbRead`, which the previous revision recorded as having *no implementation of any kind*, has two; `GuestRam` has a production implementor (`MachineRam`, in `kayfabe-qemu-raw`). `Device` **is the one port on this diagram whose only implementors are still test types** — the QEMU shell reaches the core through `GuestRam` and the register plane instead.

4.2 The crate dependency graph

Figure 2 is derived from every `Cargo.toml` in the tree and cross-checked against `Cargo.lock`; it is not drawn from the design documents. Three findings come straight off it.

The external dependency surface is still essentially zero, and that survived a 33% growth in members. In the whole workspace there is exactly one non-dev external crate: `libc`, a real dependency of `kayfabe-linux-raw` only, and a *dev*-dependency of `kayfabe-vmm-kvm` and `tests` for asserting exact errnos — plus `trybuild` and `proptest`, both dev-only. **Twenty-three of the twenty-four** members have no non-dev external dependency of any kind. *Six crates were added since the last pin and not one of them added a dependency*, including the two that talk to QEMU and the one that talks to the host driver. Whatever else is true, the supply-chain surface is not a place to look for problems.

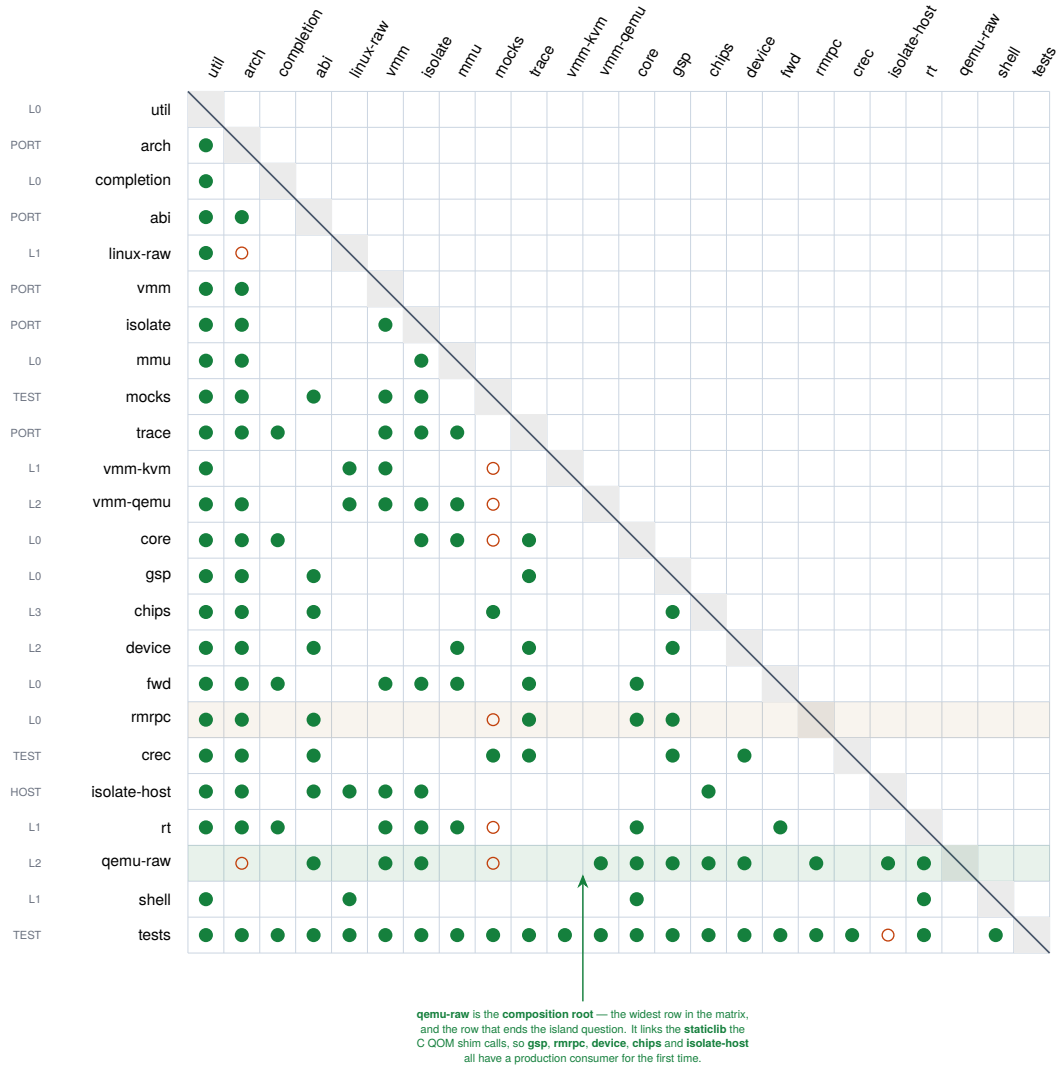
The graph is acyclic, including dev edges, which the strictly lower-triangular shape of the matrix demonstrates directly. *The ordering that makes it lower-triangular had to change again*, for the third revision running, and this time by more than a row: **18 members became 24**. The new rows are the bands the previous matrix could not draw because they did not exist — `vmm-qemu`, `qemu-raw` and `device` (L2), `chips` (L3), `isolate-host` (the host worker) and `crec` (the C-trace harness).

But the claimed strata are not disjoint, the documents imply they are, and the newest rows make it worse rather than better. `kayfabe-mmu` — a core crate — depends on the `kayfabe-isolate` *port* (it needs `HostHandle`). `kayfabe-core` and `kayfabe-fwd` depend on the `trace` and `vmm` ports. `kayfabe-gsp` — also a core crate — depends on `abi` and `trace`. And now `kayfabe-device` (L2) depends on `kayfabe-gsp` (L0) while `kayfabe-chips` (L3) depends on it too. "Core", "port" and "layer" are one DAG with a purity rule, not three tiers. That is a defensible design, but a reader who takes the hexagon picture literally will be surprised by the manifests.

★★ **This paper has now been wrong about the island question three times, and the fourth answer finally closes it.** Revision one called `kayfabe-abi` and `kayfabe-gsp` "disconnected islands". Revision two sharpened that into its headline — *"the critical path is the bridge, not the crate"* — and revision three recorded that the bridge had been built and **the island had simply moved up one crate**: *"the only manifest that lists* `kayfabe-rmrpc` *is* `tests/Cargo.toml`*."* **That sentence is now false too, and this time the property does not move anywhere.**

Re-measured against the manifests, not inherited. `kayfabe-rmrpc` is named by `crates/kayfabe-qemu-raw/Cargo.toml`, and `kayfabe-gsp` by *six* production manifests (`chips`, `device`, `qemu-raw`, `completion`, `crec`, `rmrpc`). `kayfabe-qemu-raw` is a `staticlib` that a C QOM shim links into a real QEMU binary, so the chain bytes → FSM → `RpcCommand` → `RmEvent` → `Gpu::apply` now runs inside a hypervisor a guest driver is talking to. **The island question is answered, and answering it took three revisions and one commit each time to invalidate the answer.**

★ **What replaces it, stated so it can be checked the same way.** The successor property is no longer about *reachability* — everything is reachable now — it is about *arrival*: **on every boot to date the device reports** `doorbells: 0 arrived`. The bridge is called, the register plane answers, 92–95 commands decode per boot, and the execution plane below it has never been asked for anything by a guest. That is a sentence a single boot can falsify, which is the only kind worth printing here.



Reading it. Row r, column c filled $\Rightarrow$ crate r depends on crate c. All 126 `[dependencies]` edges (●) and all 9 `[dev-dependencies]` edges (○) that are not already hard edges are shown; every `kayfabe-` prefix is dropped. `fuzz/` is a separate workspace and is excluded, as in both previous revisions.

Strictly lower-triangular in this ordering $\Rightarrow$ the graph is a DAG, dev edges included. **18 members** $\rightarrow$ **24**, and the ordering had to change again: six crates are new, and they are the L2, L3 and host bands.

External dependencies, in total: `libc` — a *real* dependency of `linux-raw` only, and dev-only in `vmm-kvm` and `tests` — plus `trybuild` (dev-only in `isolate` and `linux-raw`) and `proptest` (dev-only in `tests`). **Twenty-three of the twenty-four** members have no non-dev external dependency at all. *Six crates were added and none of them added a dependency.*

The claimed strata still do not survive, and the newest rows make it worse: `mmu` (core) depends on the `isolate` *port*; `core` and `fwd` depend on `trace` and `vmm`; `gsp` and `rmrpc` (core) depend on `abi` and `trace`; and `device` (L2) depends on `gsp` (L0) while `chips` (L3) depends on it too. "Core", "port" and "layer" are one DAG with a purity rule.

Figure 2: The crate dependency matrix, re-derived from scratch by script from the manifests at `f33e1c8` — **24 members**, 126 `[dependencies]` edges and 9 `[dev-dependencies]` edges that are not already hard edges. `kayfabe-qemu-raw` is the row that matters: it is the composition root, the `staticlib` a C QOM shim links into a real QEMU, and it is what gives `gsp`, `rmrpc`, `device`, `chips` and `isolate-host` a production consumer for the first time. The previous revision's matrix is superseded rather than corrected — it was accurate for the 18 members it drew.

4.3 The data path

Figure 3 traces one guest GPU operation from a CUDA call to real silicon and back, naming the symbol each step lives at. Three things about it are worth stating in prose.

The plan/execute/commit shape is the whole concurrency design. A host RM verb can block for a long time — the host RM serializes every ioctl per client, and it waits *uninterruptibly*, so one wedged verb parks every isolate that shares the client in D-state. Invariant **R1** therefore forbids issuing a verb while any ranked lock is held. That forces a three-phase shape: *plan* under locks, *execute* with no lock held, *commit* after re-acquiring. And because the locks were dropped in the middle, invariant **R5** requires the commit to re-validate everything the plan assumed. This is not free — §8 covers what it costs.

The ring gate is the #14 fix, and it is now structural in the type system. Before any host operation, a doorbell submission's entire working set must already be published into *this* `Vas`'s own host address space. If it is not, the call refuses with `NoVas` having issued zero host operations. The failure it prevents is exactly the one the C artifact hit on real hardware: the losing process's identical guest VAs were never published into its own host GR address space, so the host GPU faulted past the shared prefix. At the previous pin this was a call-graph property — "every path happens to go through `plan_doorbell`". At `634b253` it stopped being one: `VerbPlan::Doorbell` is `#[non_exhaustive]`, so no struct expression for it exists outside `kayfabe-isolate`, and the sole constructor `VerbPlan::gated_doorbell` *runs* the gate. Constructing an ungated doorbell is now compile error E0639, pinned by a `trybuild` row (`crates/kayfabe-isolate/tests/ui/`).

The upper half of the diagram is now fed by a real guest, and what remains missing has moved from the top of the diagram to the middle. Three revisions ago there was no register model, no GSP transport and no RPC decode; two revisions ago all three existed and nothing consumed them; one revision ago the bridge consumed them and nothing fed the bridge. **That last gap is closed**: `kayfabe-qemu-raw`'s `MachineRam` is a production `GuestRam`, a C QOM shim links the crate into a real QEMU, and a stock NVIDIA driver's own writes reach `GspFsm` — 92 to 95 commands decode on every boot. The sentence that survived three revisions — *"no guest driver has ever posted to any of it"* — is **retired**. Its replacement is narrower, sharper, and is the shape of the whole remaining problem:

> **A guest drives the control plane. No guest has ever reached the execution plane.** Every boot to date reports `doorbells: 0 arrived`. Steps **5–9** of Figure 3 — route, gate, plan, execute, silicon — have been driven end to end on real hardware and have moved real bytes with a real copy engine, but they were driven by a test harness handing the core a guest-shaped GPFIFO entry, *not* by the guest in the top lane. The guest dies in `RmInitAdapter` at step 0. **The diagram is now two working halves that have not been joined**, and §8.2 is the account of what stands between them.

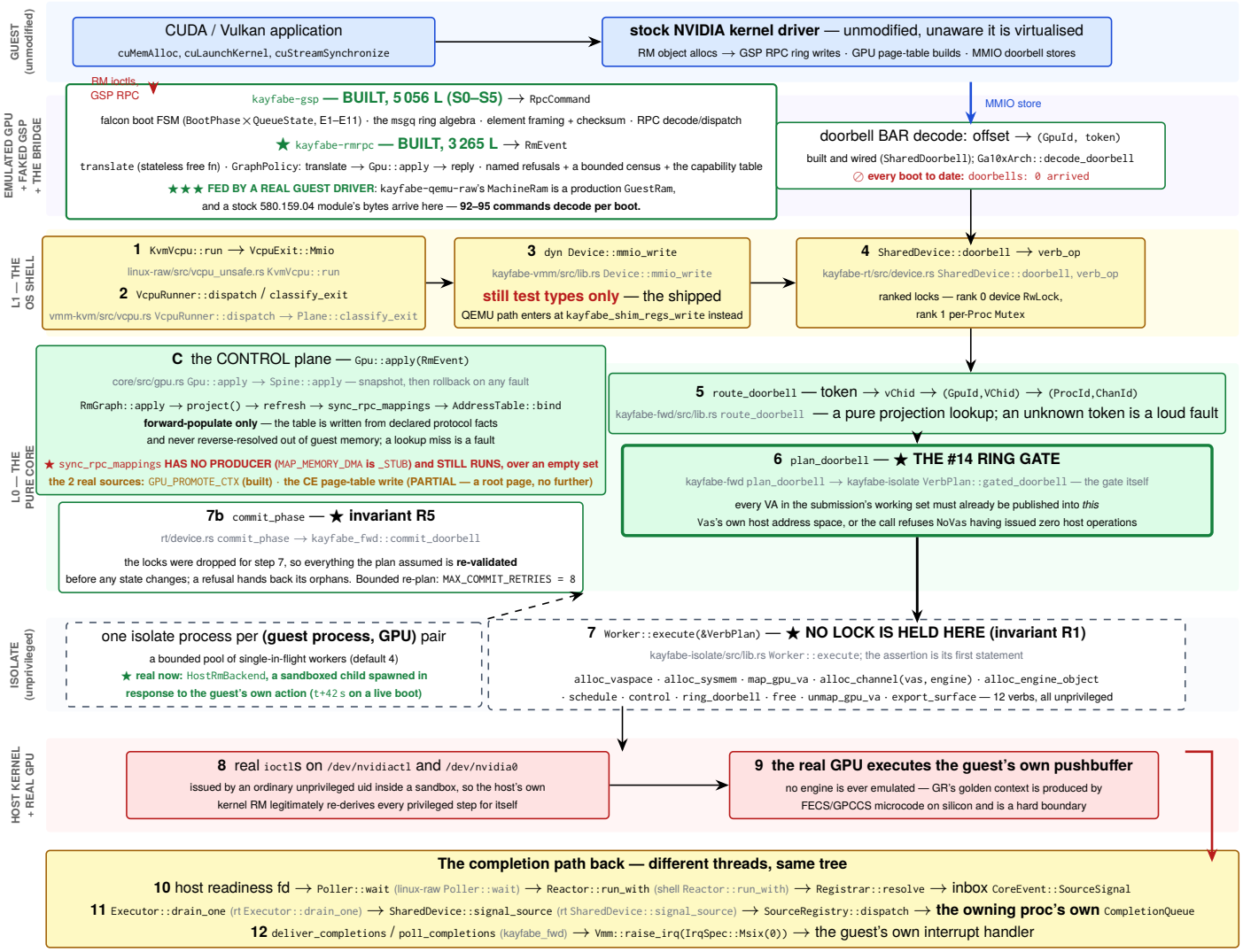


Figure 3: The data path, verified function by function against the tree at `f33e1c8`. Numbered steps are the execution plane; **C** is the control plane, which populates the address table from declared protocol facts. Boxes name *symbols*, not line numbers: the repository converted its own 105 pins to symbols in `3fd1455` after they drifted. **What changed at this revision is that most of the red went green** — the guest band now really does feed the emulated-device band, the isolate band has a real `RmBackend`, and steps 7–9 have executed on silicon. **The red that is left is the joint**: the doorbell box, which is built, wired and has never been rung by a guest. Red text marks what does not exist or has not happened.

4.4 The isolation model

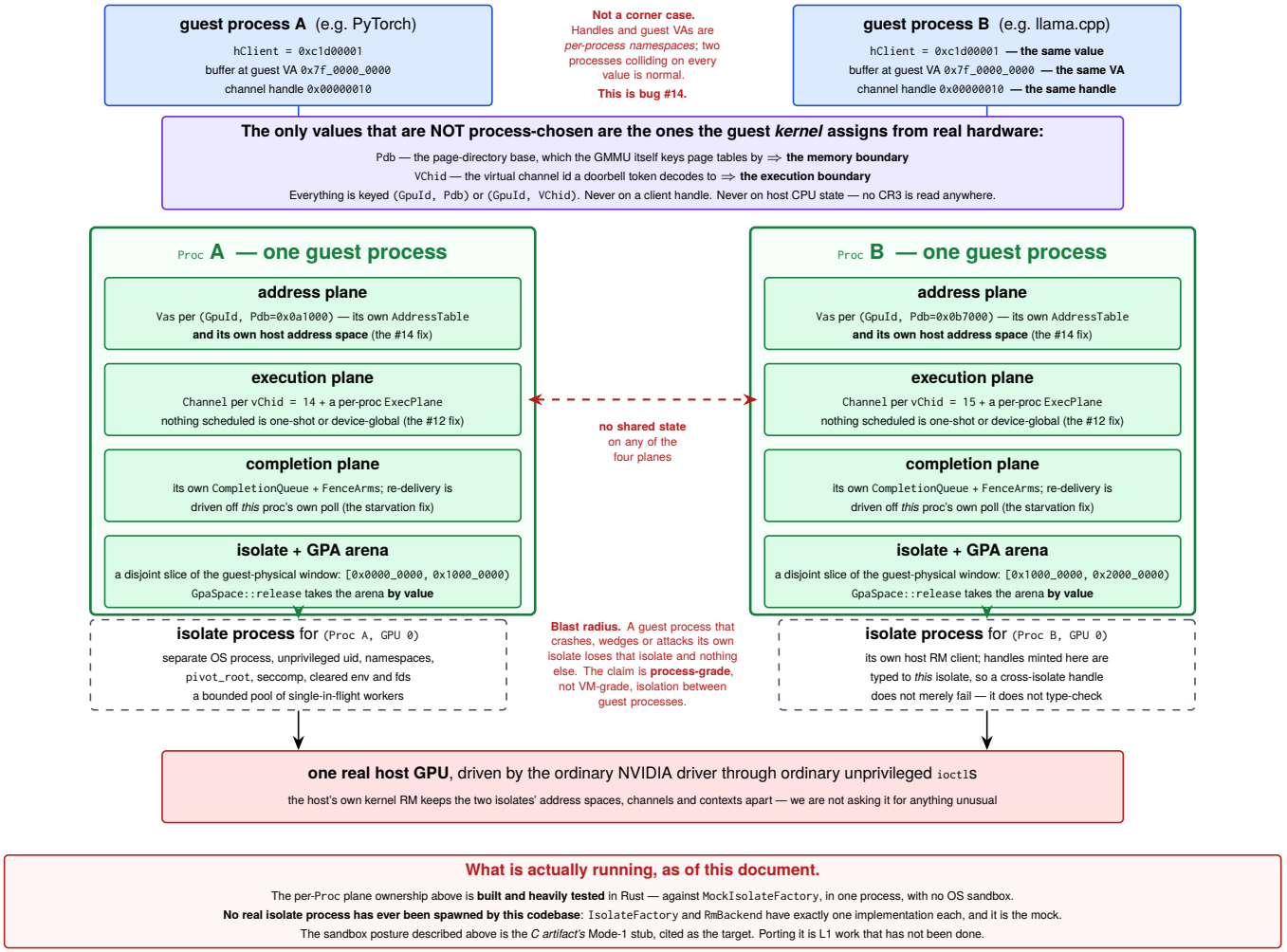


Figure 4: The process and isolation model, and the problem it exists to solve. The per-Proc ownership of all four planes is built and heavily tested against mocks. ★ **The sandboxed isolate process at the bottom is now real** — capability-less, six fresh namespaces, holding `/dev/nvidiactl` and `/dev/nvidia0`, spawned in response to the guest's own action. ⊘ **But the multi-Proc property this figure exists to argue is *unmeasured* on hardware**: every bench arm to date has exactly one Proc, because the guest boot never reaches a second guest process. The picture is proven against mocks and exercised on hardware one Proc wide.

The rule that generates the whole model is one sentence: **a guest process is a grouping label, and it is never what an address or execution operation keys on.** Address operations key on `(GpuId, Pdb)`; execution operations key on `(GpuId, VChid)`. `Proc` exists only to own the isolate, the GPA arena and the lifecycle.

`Proc` membership is itself a **pure projection** of the RM graph — the dup-connected components of declared user clients — never inferred from timing, never from arrival order. That matters because it means two identical event streams delivered in different orders produce bit-identical process boundaries, which is a property the test suite checks directly by permuting streams.

4.5 The lifecycle

The law every path obeys: RETIRE EAGER, REAP DEFERRED

Retire is immediate and cheap: the `Proc` stops accepting operations, its isolates refuse new check-outs. **Reap** is the heavy half — freeing host objects, killing isolate processes, recycling the GPA arena — and it runs later, at a quiesce point the adapter declares (`Gpu::reap_retired`).

Why: the C artifact proved that reaping at the client-root free *hangs the dying context's own residual polls*. Reaping never is the #80 leak. Both were paid for on hardware.

TRIGGER	WHAT HAPPENS	STATUS — read from the tree
T0 a guest frees a subset and keeps running	The most frequent path in a real workload, and **the one with no backstop** — nothing dies, so nothing reclaims. Dropped host identities move to a `pending_release` queue on the `Proc` and drain on a checked-out worker.	**Built, and lazier than designed.** The drain may fire only when the isolate is otherwise idle (`is_quiesced`), because no lock can exclude a verb that runs lock-free.
T1 a verb is cancelled mid-flight	The cancel is latched, discharged with no lock held, surfaces as `EINTR` → `Interrupted` → `FwdFault::Cancelled`; the chain's own unwind frees what it had allocated; the worker is checked back in.	**Built and tested** (`tests/tests/cancellation.rs`).
T2 a guest process exits — normally, killed, or killed mid-verb	All three are one path: the guest kernel frees the client root on fd close, so an `RmEvent` arrives, the projection finds no boundary, and the retire is **graph-driven**. Every checked-out worker is cancelled; fresh handles become `Orphans`; the `Proc` lands on the retired list awaiting T3.	**Built and tested.**
T3 the GPU goes idle — THE quiesce point	The guest *tells* us: its driver emits `UNLOADING_GUEST_DRIVER` (RPC fn 47) on an idle release and re-runs the queue handshake. That re-handshake is the quiesce point, and the C already ran its deferred reap there. Same doctrine as the address table: an observed protocol event, never an inference about idleness.	**HALF A LAW.** `reap_retired` exists and `CoreEvent::Deferred(DeferredReap)` exists — **and nothing arms either**. Reap-deferred with no trigger is indistinguishable from reap-never in a run that never quiesces.
T4 the guest driver restarts / the GSP re-handshakes	On `rmmod`, guest panic or VM reset the guest sends *no* `Free` events, so the graph, every live `Proc`, its isolates, arenas, host handles and routing maps all survive into the next driver life — which then derives a second set of components beside the corpses.	**NOT BUILT, and the obvious trigger does not exist.** Bench-measured 2026-07-26: `rmmod` emits **no fn-47 at all** — the idle release at process exit already consumed it. **The unload has no observable signal.**
T5 an isolate worker dies	Routing is settled (the component is condemned, keyed on its client set). Reclamation is not: a worker that died *mid-chain* left allocations that are in no `Orphans` and in no core state. The answer is to escalate to isolate death — `SIGKILL` — because killing the process is how you reclaim state you cannot enumerate.	**Partly built.** The design calls this "the single most leak-prone moment"; the carrier type (`VerbFailure{err, orphans}`) exists.
T6 isolate garbage collection	`SIGKILL` → the process's *exit descriptor* becomes readable → reactor → `waitpid` (non-blocking) → close descriptors, tear down every double-mmap, release arenas, drop worker slots. The kernel already released the host RM objects by closing the connection.	**Designed; the OS half is still unexercised** — but **real isolate processes ARE spawned now** (`HostIsolate`, a sandboxed child, first sighting `t+42 s` on a live boot). What is unexercised is the *reaping*: no boot has reached a teardown.
T7 whole-device teardown	An explicit ordered `shutdown()`, never a `Drop`: stop accepting traps, cancel every worker, wait bounded, retire everything, reap unconditionally, kill every isolate, drain the inbox asserting every remaining event faults rather than mutates, join the threads with no lock held, and **assert the conservation ledger balances**. `Drop` is a tripwire that logs loudly if `shutdown()` was skipped.	**Designed; partly built.**

The instrument all eight are measured by — the conservation ledger `HostLedger`, `kayfabe-mocks/src/lib.rs:1241`

Replays the recorded verb log and demands: every acquisition matched by exactly one release, nothing released that was never acquired — objects *and* mappings.

A test may declare a `ResidueClaim` for what it knowingly leaves outstanding, and the counts are compared for **equality, not as a bound**: less residue than declared fails too, so a fix that closes a leak must delete the claim that excused it.

★ **Its first census over the mean run reported zero leaks — and that was a true negative, not a pass.** The workload it was measuring only ever added and removed *RPC* bindings, which own nothing host-side.

Once a phase was written that actually allocates before it frees, the honest baseline was **24 objects, 6 mappings and 24 576 GPA bytes leaking, linear in frees.**

Figure 5: The eight teardown triggers and the retire/reap split. Two of the eight are honestly broken and say so: T3's quiesce trigger is designed but nothing arms it, and T4's obvious trigger was measured on hardware and does not fire.

The lifecycle is where the C artifact's scar tissue is thickest, and it is worth understanding why the split exists. Reaping resources at the moment the guest frees its client root *hangs the dying context's own residual polls* — bench-proven. Not reaping at all exhausts the shared guest-physical window after a handful of process generations — also bench-proven, as bug #80, and *re-introduced and re-found in the Rust core* by the regression test written for it. Hence: retire eagerly (refuse new operations, cheap), reap later at a quiesce point the adapter declares.

Figure 5 marks two of the eight triggers red, and the reasons are different in an instructive way. **T3** is a design that is not wired: `reap_retired` exists, `CoreEvent::Deferred(DeferredReap)` exists, and nothing arms either. **T4** is worse — the trigger the design chose *was measured on hardware and does not fire*. The design assumed `rmmod` emits RPC function 47; the bench showed it emits none, because the idle release at process exit already consumed it. The problem is not that two triggers are indistinguishable; it is that the driver unload has *no* observable signal.

5 The crates

Sizes are `crates/<name>/src/**/*.rs`, measured at `f33e1c8`. "Maturity" is this document's reading of the tree. ⚠ **It is no longer cross-checked against** `docs/design/crate_maturity_map.md`**, because that document is now wholly stale** and this paper will not launder a number through it: it is pinned at `d65eb75` ("496 tests green"), lists **16** crates where the tree has 24, calls `kayfabe-gsp` a **34-line skeleton** where it is 5 056 lines, and records defect #57 as open five days after it was closed. It is a worked instance of §8.16 and is listed there (§10 row 29).

The tree grew by 2.7× and gained six crates since the previous pin. The six are the reason §8.14 and §8.2 read differently from every previous revision: `kayfabe-device`, `kayfabe-vmm-qemu` and `kayfabe-qemu-raw` are the L2 adapter this paper twice described as not started; `kayfabe-chips` is the L3 adapter whose contract had never been tested by anyone doing it; `kayfabe-isolate-host` is the real unprivileged host worker; and `kayfabe-crec` is the C-trace differential harness.

5.1 `kayfabe-core` — 10 528 lines, **BUILT** and mutation-hardened

Owns the spine. `RmGraph` is the source of truth: a refcounted resource/handle split with `DUP_OBJECT` aliasing, order-tolerant "parked" facts (a mapping that arrives before its target is held, not rejected), mapping refcounts and capacity bounds. `project()` is a pure function from the graph to process boundaries and the two routing tables. `Gpu::apply` is transactional — snapshot, mutate, re-project, roll back on any derivation fault, so a hostile event earns only its own refusal. `gpa.rs` owns the guest-physical window and carves per-process arenas; `GpaSpace::release` takes the arena *by value*, which makes releasing a live process's arena unrepresentable rather than merely wrong.

Deliberately not here: anything about wire formats, any NVIDIA constant, any OS call.

The sharpest single defect the project has found in its own core is still the one described in §8.5 — a live-memory-free in `RmGraph::apply` whose independent oracle carried the identical bug. Nothing since has displaced it, and the parked-fact machinery that produced it is still where the difficulty in this design lives.

5.2 `kayfabe-abi` — 28 630 lines, **BUILT** — now the largest crate in the tree

Axis A: the per-driver-version wire layouts. Contains an offline generator (`crates/kayfabe-abi/gen`, a separate cargo project with zero third-party dependencies) that parses NVIDIA's own FINN-emitted C headers from the open kernel modules and emits Rust; 13 generated structs plus one hand-transcribed variant; a version-dispatch table keyed on full `major.minor.patch`; and a large oracle suite. The `#[repr(C)]` mirrors are a *layout oracle only* — decode is `from_le_bytes` out of a byte slice, never a cast, so `forbid(unsafe_code)` still holds.

Its test design is the best in the repository: layouts are pinned against **three independent oracles** — NVIDIA's own headers, gVisor's independent transcription, and the C artifact — with a `file:line` citation per number, and a test whose name is `nvos46_the_three_oracles_disagree_and_here_is_exactly_how`. That test found that the C artifact's *parity test* was weaker than the C code it was supposed to guard.

It tripled in size and acquired the two mechanisms this revision leans on hardest. **First,** `capability.rs` — the guest-ingress allowlist ported from the C's nvproxy-derived table: 162 control rows and 91 allocation classes at 580, each carrying NVIDIA's own name and an `Origin`, six controls and two classes refused *by name*, resolved denial-first (which is stricter than nvproxy). **★★★ And "default-deny" is true of half the command space, not all of it**, which the module says in its own header: the GSS-legacy rule passes every command with bit 15 set — 2^{31} values, no table row, no review — because that is nvproxy's rule and the C's, and this is a port. A test named `the_gss_legacy_rule_passes_half_the_command_space` pins it. ⊘ It also does *not* carry the host-side

security property: all six denied controls are `NON_PRIVILEGED` in RM, and deleting the table tomorrow would leave that property standing (§8.13).

Second, `oracle.rs` **and** `fmbsize.rs` — the machinery that refuses an uncaptured C oracle row, which is §3.1's fifth limit turned into a predicate rather than a list.

It now has six production consumers, and it is still the crate that keeps every NVIDIA constant quarantined: the bridge names none, by design and by review. *The previous revision's surviving criticism is now expired too*: it said the version-dispatch machinery was "a built capability with a test-only interrogator" because `gsp_element_wire()` had no caller in `crates/*/src`. It has one now — the shipped register plane selects a wire layout to answer a real guest through.

5.3 `kayfabe-linux-raw` — 12 470 lines, **BUILT** — the *first* of two `unsafe` crates

One of exactly two crates permitted `unsafe`, and one of the two that do not inherit the workspace lints (because `forbid` cannot be relaxed by an inner `allow`). It restates `unsafe_code = "allow"` plus `clippy::undocumented_unsafe_blocks = "deny"`. Every file that uses the keyword is named `*_unsafe.rs`, so `ls` enumerates the audit surface: 12 files, **87 relaxations**. It also carries a `trybuild` compile-fail matrix covering things like a bare `MAP_FIXED` address, a page-size literal, and a view outliving its region.

★ **The ratchet moved from a constant to a named two-element list, exactly as §8.14 predicted it would have to** — `AUDITED="kayfabe-linux-raw:87 kayfabe-qemu-raw:47"` — and the acceptance criterion the gate protects is now "read two crates". The movement is itemised per step in the workflow with an argument each: $7 \to 25$ (first consumers) $\to 37$ (a reactor and a guest) $\to 47$ (a host GPU and a child process) $\to 50$ (a machine it did not create) $\to 57$ (**closing a measured host-filesystem escape**) $\to 60$ (closing the privilege half) $\to 73$ (the embedded isolate) $\to 87$.

> ★★★ **And the ratchet's own counting pattern was wrong, found by being its first FFI consumer.** `unsafe ({|fn |impl |trait )` **cannot match** `unsafe extern "C" fn` — the dominant form in a foreign-function crate. **It counted 23 of 31 and called it a complete audit.** The fix was not to bump the bar: `kayfabe-linux-raw`'s count was *re-measured* under the corrected pattern rather than carried forward, because *"it was 57 when I looked" would have been a claim about a tree that no longer exists.* ★ A second rule came out of the same review: **thirteen relaxations are the surface, not a style** — a macro emitting the call would collapse all thirteen into one textual relaxation, because the ratchet counts source tokens and a macro body is one token. *That is gaming the instrument.*

5.4 `kayfabe-gsp` — 5 056 lines, **BUILT** (S0–S5) — **and a real guest driver now writes into it**

The faked GSP: the guest driver's entire picture of its GPU arrives through this crate. Eight modules. `ram` owns the guest-RAM port and walks the shared region's *self-describing page table* (the region is allocated `NV_MEMORY_NONCONTIGUOUS`, so the C artifact's `base + offset` arithmetic is a latent bug this crate does not reproduce). `ring` is NVIDIA's `msgq` algebra — geometry, cursors, and the `msgqRxLink` acceptance predicate reproduced code-for-code so "would the guest link this header?" is answerable offline. `element` is framing, the 64-bit XOR-fold checksum and multi-element split/join. `boot` is `BootPhase` × `QueueState` with transitions E1–E11, and it deletes all three of the C's one-shot latches — two by construction, one by making the queue GPA exist only inside a `Bound` variant. `rpc` classifies functions into three dispositions (must-reply-synchronously, reply-expected, must-*not*-reply). `replay` is the differential projection its oracle compares, plus a ledger in which a deliberate divergence is admissible only with four things attached, one of them an independent oracle.

Five things it is deliberately not tied to, each with a test that pushes on it: a GPU architecture (every offset is behind `kayfabe_arch::GspModel`; the suite drives the same FSM through two deliberately-different models), a driver version (the element layout is an `ElementLayout` *value*, not a constant), a hypervisor (the only way in is an abstract register access and the `GuestRam` port; CI greps for every VMM type name), a single GPU lifetime (`GspFsm::device_reset` rebuilds the value and teardown drops the queue binding *by value*, so driver-restart works in-process, repeatedly), and a CPU architecture (every wire access is explicitly little-endian).

What changed underneath it at this revision is that it stopped being a model. It has **six production consumers** (`chips`, `device`, `qemu-raw`, `completion`, `crec`, `rmrpc`), and — the part that is not a manifest fact — a stock NVIDIA 580.159.04 kernel module in a KVM guest now writes into this FSM through a real BAR. Its boot phases are driven by a driver that does not know it is being driven. **GSP-S1**, the guest-kernel memory-safety invariant an earlier revision of this paper singled out as "the most security-relevant thing in this document that is a sentence rather than a mechanism", remains a mechanism and is now one a real guest is on the other side of. §8.7 is about what remains.

5.5 `kayfabe-rmrpc` — 3 265 lines, **BUILT** — **the bridge, and no longer an island**

Named by `crates/kayfabe-qemu-raw/Cargo.toml`, which is what retires the previous revision's headline about it. Its whole entry point is still one signature:

```
translate(&DriverAbiTable, &RpcCommand) -> Result<Translation, BridgeRefusal>
```

and it is a **free function** — no `&mut self`, no handle table, no memo, no seen-set, no guest-memory access and no lookup. That statelessness is the design's central claim rather than a tidiness preference: a per-handle cache here would re-open the whole recycled-handle bug class the core closed the same week, so *recycle-safety is structural rather than tested*. The suite pins it by mutation — a per-handle memo and a seen-set are planted as deliberate mutants at every stage, and by B3 they were visible to 20 and 24 tests respectively.

`policy.rs` holds the only thing in the crate that applies: `GraphPolicy`, the `kayfabe_gsp::CommandPolicy` the boot FSM calls, whose `respond` is translate → `Gpu::apply` → reply. A refusal answers with a non-zero status and an **empty** body — deliberately not the request echoed, because reflecting guest bytes under a failing status is `memcpy(resp, cmd, 4096)`, the C defect this deviates from by name. Refusals are counted by a `RefusalCensus` bounded by the finite tag set rather than by traffic, with `applied()` and `inert()` kept as *two* counters so a regression that turned every alloc inert cannot hide inside a sum.

What it deliberately is not: it is not in `kayfabe-fwd`, where both the port plan and `rpc.rs`'s own comment said the bridge would go — `fwd` depends on neither `abi` nor `gsp`, and B1 needed no `Worker`, `Vmm` or `RmBackend` at all. Three superseded placements are recorded in the design doc rather than quietly dropped.

Its honest limits, stated as the crate states them: it **still cannot emit a** `Translation::Forward`, because the control-classification table it would hand to does not exist, so the entire control long tail is refused by name; and it cannot reach the compute path at all (§8.4). ★ That second limit is quietly load-bearing for security: three descriptor-passing controls that a multi-tenant audit says should be denied are still on the allowlist, and the only thing stopping them mattering is that **the forward arm is unwritten** — *which is a schedule, not a control* (§8.13).

5.6 `kayfabe-rt` — 3 009 lines, **BUILT** (L1-M1)

The threaded shell. `LockRank{Device=0, Proc=1, Leaf=2}` with an always-on (not `debug_assert`) rank check that panics *before* the OS acquire, so an inversion is deterministic. `SharedDevice` is the plan/execute/commit driver, and it ships **both** lock modes — `Sharded` and `Degenerate` — from day one, tested as first-class configurations with a differential asserting a bit-identical end state, specifically so a late granularity change is never the untested mode.

5.7 `kayfabe-fwd` — 4 490 lines, **BUILT** — and **part of it has now moved bytes on silicon**

The entry points an adapter calls: `handle_doorbell`, `publish_backing`, `parse_pushbuffer` (the one parser: CE page-table-write capture, semaphore release, TLB invalidate, everything else opaque and passed through), `forward_engine_object` and `route_control` (the Case-1/Case-2 split), the fence arms, and the present route.

It gained the E6 join, which is what changed its epistemic status. `plan_ce` / `commit_ce` / `exec_ce` / `submit_ring` are the three R1 phases for a partitioned copy request plus their composition. Before that, `PushbufferOutcome::ce_spans` — the partitioned request the parser had produced since #102 — **had no caller anywhere in the workspace**: every copy the core recovered from a guest ring was computed, asserted about, and dropped. `commit_ce` adopts nothing and is **never retryable**, because re-running a copy that executed performs it twice.

The honest scope. A real host copy engine has executed a request this crate decoded and planned (§8.2). **That is one path.** The doorbell path has been driven to `UnknownVchid` on a boot and to `IsolateRetired` in-process, and never further. The control long tail, the fence arms and the present route remain claims about NVIDIA's behaviour that nothing here has checked against NVIDIA.

5.8 `kayfabe-mocks` — 4 172 lines, **BUILT**, test-only — **and where an invariant hid**

One deterministic fake per port, plus the shared verb recorder and the conservation ledger. Excluded from the mutation denominator on the argument that mutating an instrument is a category error. *Strictly*, the crate is not confined to dev-dependencies: `tests/` and `fuzz/` list it as a normal dependency, so `cargo build --workspace` compiles it. Nothing shipping links it.

> ★★★ **And this crate is where invariant R1 hid for an unknown number of commits.** The assert that guards R1 at an isolate spawn lives in `kayfabe-linux-raw`, at the syscall — so it is reachable *only on the host plane*. The whole

suite spawns through `MockIsolateFactory`, whose `spawn` blocks on nothing and asserts nothing. **1 975 green tests ran over a live R1 breach**, and only a real boot on a real GPU found it (§8.12). The lesson, stated generally in the design record: *an invariant asserted only in the production adapter is an invariant the test suite does not hold. Where the mock and the real implementation differ in* ***whether they can violate a rule****, the assert belongs at the seam they share.* It was moved there — `IsolateBox::new` now asserts, which turns the entire mock-driven suite into a live guard for it.

5.9 `kayfabe-vmm-kvm` — 2 706 lines, **BUILT** — **defect #57 CLOSED**

The real KVM memory plane: real `/dev/kvm` file descriptors, real memslots, real vCPU run loops, MMIO exit classification. **Defect #57 — which this paper twice printed as the reason the QEMU adapter had not been started — was closed at** `020790c`, and the shape of its resolution is the part worth keeping: *the violation was not the two-lock split, the lock order, or a syscall inside a critical section. It was* `Arc` *ownership* — a reader's clone became the last one and `munmap` ran on a thread legally holding rank 0. **R1 is about blocking calls, and a** `Drop` **is a call site.**

5.10 `kayfabe-isolate` — 2 666 lines, **BUILT** traits — **with real implementations now**

The port for the unprivileged host worker: `RmBackend` (12 verbs, all of which must be issuable unprivileged), `Isolate` (two-stage retire, cancellation, quiesce predicate), `IsolateFactory`. A `HostHandle` carries its `IsolateId`, so a cross-isolate handle does not merely fail — it does not type-check. *The previous revision's headline for this crate — "no real implementation" — is retired by* `kayfabe-isolate-host` *below.* And `Worker::execute` is no longer "the single door": `IsolateBox::new` and `IsolateBox::drop` are now R1 doors too, closing birth and death of an isolate respectively.

5.11 `kayfabe-isolate-host` — 9 939 lines, **BUILT** — **the real unprivileged host worker**

New, and it is what makes half of this paper's hardware claims possible. A sandboxed child process: capability-less, six fresh namespaces, holding `/dev/nvidiactl` and `/dev/nvidia0`, issuing real host RM ioctls. It carries three binaries — `kayfabe-isolate` itself, `kayfabe-rm-ladder` (a human-facing bring-up ladder, deliberately *not* part of the isolate, because the isolate faces a hostile guest), and `kayfabe-sandbox-probe` (a separate *process* by necessity: entering the sandbox rewrites this process's mount namespace and root, so the containment property can only be observed from outside). `CeEvidence::copied()` — the predicate the whole execution plane is accepted against — lives here.

★ **Building it closed a measured host-filesystem escape**, which is why the unsafe ratchet moved 50 → 57: the isolate's own `/dev` descriptor could open `../etc/shadow` on a real host. That is a defect the C artifact had found, fixed and documented, and the Rust rewrite reproduced before re-fixing it — a worked example of *port the C and subtract its named bugs* failing on a bug that was named.

5.12 `kayfabe-trace` — 1 470 lines, **BUILT** vocabulary; **still not threaded**

The typed `TraceEvent` vocabulary, a `TraceSink` port, one-counter total ordering, perf-budget counters and a projection differential. It exists because the GSP milestone's oracle *is* trace replay, so the replay format had to be defined before the harness that consumes it. Two findings from building it are worth carrying: it had to sit *below* `kayfabe-core` (every plane must be able to emit), so it cannot name `RmEvent` or `ProcId` — the bridge is a trait the owning crate implements with an exhaustive `match`, so a new variant upstream fails the build until the vocabulary names it. And no `&mut Trace` is threaded through any plane signature yet; the vocabulary is driven only from the conformance suite's seam observer.

5.13 `kayfabe-shell` — 995 lines, **BUILT** (L1-M2, in progress)

The reactor: an epoll loop, source resolution, the inbox, and the executor's wake. It carries the restated F1 rule — *assert a bound, never an absence* — as a concrete quantity: after 16 consecutive unproductive wakes it refuses with `ReactorFault::UndrainableSource` rather than spinning. That restatement came from the mutation gate: three spin-versus-park mutants survived precisely because "never busy-poll" is an absence, and an absence is not observable.

5.14 `kayfabe-vmm` — 914 lines, **BUILT** traits — **and the agnosticism claim is now settleable**

The hypervisor and display ports: `Vmm` (7 capability groups), `Device`, `Present`. An eighth group — the memory-lock primitive — was *removed* from the trait when it turned out to need no hypervisor cooperation on any backend, which is a good sign about the seam's discipline. `Vmm` **now has two real adapters** (`KvmVmm`, `QemuVmm`) with a differential test asserting they produce identical operation logs for the same core run, which is the

experiment the previous revision said only a second backend could run. [unverified] here: this paper did not execute it. `Device` **remains the one port on the layer diagram whose only implementors are test types** — the shipped QEMU path enters through `GuestRam` and the register plane instead, so the trait was routed around rather than implemented, which is worth noticing before treating it as a live seam.

5.15 `kayfabe-device` — 9 832 lines, **BUILT** (L2 stage Q4) — the chip table and register plane

New. Two things and nothing else: `ChipProfile`, one row per GPU generation carrying the PCI identity, the `GspModel` constructor, the silicon-constant registers and the VBIOS aperture; and `RegPlane`, the routing of one trapped register access into `GspFsm` and back out as a value. *It exists so there is exactly one description of a chip*: before it, the only real register map lived in the trace-differential harness, which a shipped archive cannot depend on, so wiring a guest's accesses meant either shipping the mocks or writing the offsets a second time — *two descriptions of one chip that can disagree*. The consequence is that the 359 062-record `cap1` replay now runs against **the same encoder the guest reads through**. Adding a generation costs a module, a VBIOS row and one appended table entry, and `tests/chip_table.rs` proves that by *doing* it with a chip whose register map is deliberately not GA10x.

5.16 `kayfabe-chips` — 2 735 lines, **BUILT** (L3) — **the adapter contract, finally measured**

New, and it exists to take one measurement. The port plan had named the home of an `Arch` implementation as "an arch-impl crate" for months; **no such crate existed**, so *"adding a generation is* `impl Arch for <Gen>` *in an adapter crate"* had never been done by anyone. It holds two generations that answer in **opposite directions**, reported separately because averaging them would hide the interesting one. **Ada** costs a struct and a `VBIOS_PROFILES` row and nothing else — ★ and that result *"is weak on its own and is labelled as such: an experiment that selects the easiest member of its universe produces a green with no red available to it."* **Hopper** is the hard member, and it was never the offsets: the boot ordering was spelled out in `match` arms inside a *logic* crate, so a generation whose boot is driven by an FSP command queue could only be expressed by editing the ordering Ampere boots on. The ordering now lives behind `kayfabe_arch::BootSequence`, and landing a second one changed **zero lines** of `kayfabe-gsp`, `kayfabe-device` or the Ada module. *Arch-specific boot code is legitimate and has to exist somewhere; the defect was that it was unhooked.*

5.17 `kayfabe-vmm-qemu` — 5 215 lines, **BUILT** (L2) — the QEMU memory plane

New. The second `Vmm` adapter: slot management, region classification, viewer installation, tiering. Under `forbid(unsafe_code)` — all the `unsafe` is quarantined one crate over.

5.18 `kayfabe-qemu-raw` — 3 435 lines, **BUILT** (L2) — **the composition root and the second** `unsafe` **crate**

New, and it is the widest row in the dependency matrix. A `staticlib` that a 2 676-line C QOM shim (`qemu/hw/misc/nvkvm/`) links into a stock QEMU 10.2.4 tree. It owns the FFI surface — `extern "C"` entry points, raw `MemoryRegion` handles, adoption of host pointers QEMU owns — at **47** audited `unsafe` relaxations, ratcheted separately from `kayfabe-linux-raw`. Its bar started at **0** deliberately, *"so that this had to be a reviewed commit rather than a manifest edit"*. `MachineRam` is the production `GuestRam`; `Regs::object_model()` is the single composition root that the object bridge declares into and the doorbell rings.

5.19 `kayfabe-crec` — 2 030 lines, **BUILT**, harness-only — the C-trace differential

New. Decodes the committed C captures and replays them against the real `GspFsm`. Two committed traces (`cap1`, `cap1b`) are stored *decompressed* so the harness needs no decoder and no third-party crate. §3.1 is its output.

5.20 `kayfabe-completion` — 756 lines, **BUILT**

Per-process queues (pending → in-flight → awaiting-ack → ack), a drain-gated delivery plane whose re-post is driven off *the owner's own poll* (the starvation fix), and mapped-fence arms with the #12 backwards-jump guard. Scored 100% on the mutation gate.

5.21 `kayfabe-util` — 739 lines, **BUILT**

`IntervalMap` (the address table's container, loud on overlap, empty and wrap), a *virtual* `Instant` so no wall clock exists anywhere in the core, the `assert_send_sync!` compile-time gate, and the lock witness that invariant R1 asserts against. The witness lives here rather than in `kayfabe-rt` specifically so that `kayfabe-isolate`, which cannot depend on `rt`, can still assert it.

5.22 kayfabe-arch — 2 198 lines, BUILT traits — **with three production implementations**

The identity vocabulary (`HClient`, `HObject`, `Pdb`, `VChid`, `ClassId`, `GpuVa`, `Gpa`, `GpuId`, `EngineKind`) and the Axis-B port set, which grew to include `GspModel`, `UserdModel`, `PushbufferAbi` and `BootSequence`. **The previous revision's standing line — "still zero production implementations ... that remains the standing proof that a real one will need no core edits" — has been settled by doing it.** There are three (`kayfabe-chips`), and the proof came out partly favourable and partly not: Ada needed no seam, Hopper needed one, and *the seam it needed was in this crate*.

5.23 kayfabe-mmu — 3 729 lines, table BUILT; **walker built; GPGA new**

`AddressTable` is complete and is the enforcement point for the project's most-cited directive: forward-populate only, unbind eagerly, and **a miss is a fault** — no reverse resolve, no heuristic pick, no MRU fallback exists anywhere in the codebase.

★ **The previous revision's headline for this crate is retired, and this paper named the gap that closed.** It read: *"the* `walker` *module is 41 lines containing one trait and one enum, with no walk function of any kind."* The module is now 863 lines with `translate`, `translate_from_entry`, `decode_page`, `decode_subtree`, `leaf_disposition` and `populate`, and `FbRead` has two implementors. It sits beside `gpga.rs` (1 390 lines), the guest-physical address space, and `reach.rs`, the reachability-on-transition machinery.

tests/ — the conformance suite

118 160 lines of test code against 120 979 lines of implementation. There are **65** files in `tests/tests/` and **67** more under `crates/*/tests/`, plus a `Scenario` DSL in `tests/src/`. The suite is a workspace member with a `[lib]`, which is why `kayfabe-mocks` compiles as a normal dependency. ★ **Note the ratio flipped**: at the previous pin test code exceeded implementation by 16% and this paper listed that as a risk; implementation has since overtaken it. That is not obviously an improvement — the growth is almost entirely in the L2/L3/host crates, which are the least tested part of the tree.

That ratio is deliberate and the reason is worth stating, because it is the crate's answer to an oracle bug found the same week. Every bridge message is transcribed **three independent ways**: an `ogkm`-derived builder in `tests/src/rpcwire.rs` that imports nothing from the decoder, hand-written offset-annotated hex arrays, and the decoders themselves — plus a differential against the C artifact's independently-transcribed offsets. The bug that motivated it was in `tests/src/gspworld.rs`: the independent guest model derived the element count *from the element under test* instead of reading the field, so the encoder's own write was unobserved by its own oracle. `250b78c` fixed the model; `gspworld.rs` is now 1 468 lines.

6 The invariant catalogue

These are the properties the design is willing to be judged on. Where an invariant has a mechanism, the mechanism is named; where it is only prose, that is stated, because the project's own doctrine is *"prefer a mechanism to a sentence — a rule stated only in prose has no reader."*

Invariant	Statement	Enforced by
Order-independence	Everything derived is a pure function of declared protocol facts, never of arrival order. "Protocol, not trace."	Permutation/interleave/dup proptests over the whole observable end state (`tests/tests/determinism.rs`).
MISS = FAULT	Tables are forward-populated only; a lookup that misses is a loud fault, never a guess. **Refined:** a ~28-site audit found the absolute had a second, correct answer — the real rule is decided *per site*: *not yet knowable ⇒ DEFER; never knowable ⇒ FAULT*. A mapping arriving before its PDB bind defers; a handle resolving to the wrong *kind* faults. Getting it wrong is asymmetric: fault-when-it-should-defer hangs the guest, defer-when-it-should-fault is a security question.	`AddressTable` has no reverse-resolve path to call. `tests/tests/miss_taxonomy.rs` makes the taxonomy executable. *No shipped site needed a behaviour change; three* ***doc*** *claims did, two of them stating the literal opposite of their own code.*
Per-target keying	`Pdb` and `VChid` are per-GPU namespaces. Cross-GPU identical ids are legal; same-target duplicates are loud collisions.	`project()`; `tests/tests/multi_gpu.rs`.

Invariant	Statement	Enforced by
Per-`Proc` isolation (I1)	Identical guest VAs and handles in two processes reach disjoint arenas, disjoint host address spaces and disjoint isolates *by construction*.	Types (`GpaSpace::release` by value; `HostHandle` carries its `IsolateId`) plus a proptest against an injective oracle.
One gated ring path	A doorbell can only reach the host after the #14 working-set gate.	**Now structural, not a call-graph claim.** `VerbPlan::Doorbell` is `#[non_exhaustive]`, so its only constructor is `VerbPlan::gated_doorbell`, which runs the gate; an ungated one is compile error E0639, pinned by a `trybuild` row. The cardinality claim corrected in §10 is what this replaced.
GSP-S1	**We may never emit an element run wider than the guest's staging buffer.** At driver 580 the guest reads our `elemCount` field directly into an unbounded `portMemCopy` loop whose staging area is exactly `queueElementSizeMax / queueElementSizeMin` elements — 16 on the bench — with the live `msgq` metadata as the very next byte. An `elemCount` of 17 overwrites the metadata the guest is actively using; at the ring's reachable maximum it writes 188 416 bytes past a `portMemAllocNonPaged` allocation. This is a *guest kernel heap* corruption reachable from a value we choose.	★ **PAID OFF AT THIS REVISION** (`250b78c`), and it was the item the previous revision named as this project's highest-stakes prose-only rule. `max_elements` is derived from the layout, `encode_message` refuses `GspFault::ElementCountOutOfRange` before building a run, *unconditionally* rather than gated on the layout's field, and the receive side range-checks the guest's declared count and cross-checks it against the derivation (`ElementCountMismatch`), leaving the cursor unmoved on refusal. *With one honest footnote:* the originally briefed bite could not fire, because at a dividing element size the length bound already implies the count bound — the count bound is independently reachable only at a non-dividing `msgSize`, which is real guest-chosen input, and the test moved there.
Completion integrity (I2)	A completion fires only from a genuinely-armed source in the *owning* process, at most once; the system forge path is typed so it cannot reach a user process's queue.	Type-level (`Traffic::System`) plus a differential against an independent reference fence model.
Refcount soundness (I3)	A resource is alive $\iff$ it has a live handle or map reference. No premature destroy, no leak, dup survives origin free.	Proptest over deep trees with cross-client dups and interior frees.
DoS containment (I4)	`Gpu::apply` is transactional; every guest-growable table is capacity-bounded with a loud refusal, never OOM.	**Corrected, and the correction is in the design doc:** "no $O(n^2)$ path on hostile floods" is **false and was measured**. The caps bound *memory*, not *time* — see §8.

Invariant	Statement	Enforced by
R1	No blocking host verb — indeed no potentially-blocking syscall — may be issued while any ranked lock is held. Blocking calls take **zero** ranked locks.	★★★ **This is the invariant that was BROKEN on the production path, and the suite could not see it** (§8.12). `assert_lock_free` guards the backend doors, and a `Drop` is a call site — one violation was exactly an `Arc` ownership issue in a `Drop` (#57). But the spawn of an isolate is also a blocking call, it ran under rank 0, and **1 975 green tests could not reach the assert** because the suite spawns through a mock. The repair moved the assert to `IsolateBox::new` — the one door every plane shares — which turns the whole mock-driven suite into a live guard.
R3	Locks are acquired in strictly increasing rank, at most one per rank.	`check_acquire`, always-on, panics before the OS acquire.
R5	After re-acquiring, re-validate everything the plan assumed.	The four `commit_*` functions; bounded re-plan at `MAX_COMMIT_RETRIES = 8`.
F1	Three unrelated things are called F1 in this project: a fixed threat-model bug, the per-GPU collision guard, and the reactor's bounded-poll rule.	All three real; the overloading is a documentation hazard, not a code one.
GL1–GL13	The guest-memory-lock invariants.	**Documentation only. Zero occurrences in any `.rs` file.** They have no port to be enforced at, because capability group 8 was removed from the `Vmm` trait. Treat as a designed, unimplemented subsystem.

6.1 The two-layer trust model

One normative rule deserves its own space, because it decides designs and because it is the cleanest statement the project has produced:

> **Layer 1 (trap + lock) gives SECURITY against a guest that ignores the protocol. Layer 2 (NVIDIA's own synchronisation points) gives CORRECTNESS for a guest that follows it. Shared tables are NEVER authoritative for an immediate decision.** [docs/design/core_security_threat_model.md §2.1]

Neither layer is a weaker version of the other. Layer 2 alone is not security: a hostile guest can update a shared table *after* validation and *before* we act, and it can fake the handshake entirely. Layer 1 alone is not correctness: a trap says the bytes cannot change while you look; it does not say they are *finished*. A cooperating guest halfway through publishing a page-directory update has written something internally inconsistent and perfectly stable, and reading it under a perfect lock yields a perfectly-locked wrong answer.

The consequence — *a shared table is evidence, never authority* — is why the address table is forward-populated and why a miss is a fault.

And the C's measurement sharpened it in a way worth carrying. The Layer-2 edge is *whichever* edge the protocol commits at, not a favourite one. On the kernel/UVM/RM paths that edge is the TLB invalidate. On the GSP-emulated compute path it is *not*: round 6 of the #14 investigation measured **zero** occurrences of both invalidate transports there. A design that had encoded "read at the invalidate" as *the* Layer-2 rule would have had no synchronisation point at all on the path that matters most. The general rule survived; one of its instances did not.

What the C artifact does not have is Layer 1. There is no lock of any kind between QEMU and an isolate in a system that runs 22 real GPU applications at host parity. It ships a copy-once snapshot instead, and the reasoning is written out in its source. The working system to date has been running on Layer 2 alone — which is correct for a cooperating guest and is not a security property.

And the boundary points both ways, which building the GSP made concrete. Everything above is about not trusting the guest. **GSP-S1** is the mirror: the guest kernel is inside the boundary we are defending, and the faked GSP is a device that can corrupt guest kernel memory from a width *we* choose. The mechanism is fully traced in `mode2_gsp_port_plan.md` §4.6 and reduces to one sentence — at driver 580 the guest reads our `elemCount` field into a copy loop bounded by nothing but the ring, and the byte after its staging area is the `msgq` metadata it is actively using, so the corruption is immediately self-amplifying. It is the same defect class as the C artifact's unguarded `% s->q_msgcount SIGFPE`, pointed at the guest instead of at QEMU. The obvious objection — *"the guest is the victim, so it is the guest's bug"* — is answered in the plan and the answer is that it is both. **At the previous revision this invariant was written down and not enforced, and this paper called that its most security-relevant sentence-rather-than- mechanism. It is now a mechanism** (`250b78c`; see the GSP-S1 row above and §8.6 for why the bite that proved it is not the bite that was predicted). The general point survives the specific repair: the boundary points both ways, and a faked device is a device.

7 What is tested, how, and what that does and does not prove

7.1 The shape of the suite

Measurement	Value	How
Tests reported green	**2 007**	`cargo test --workspace --no-fail-fast` *with* `KAYFABE_NO_KVM` *unset*, on the RTX 3060 bench, recorded at `3ab1305` — [unverified] here, since no `cargo` was run for this revision either. *Was 651.* The progression is recorded per increment: 1 835 → 1 851 → 1 895 → 1 960 → 1 975 → 1 986 → 2 007
`#[test]` attribute sites	2 034	grep over workspace members; excludes the detached generator and the `trybuild` ui fixtures
`proptest!` blocks	16	each is one test that runs many cases
Checked-in proptest failures	**20 seeds, 4 files**	Unmoved since the previous revision. ★ **This paper said "0" two revisions ago and that was wrong when printed** (§10 row 22)
KVM-gated tests	**56 reached**	up from 36; CI floors the *reached* count
GPU-gated tests	**1 reached**	the E6 hardware acceptance. One test, and it is the one that carries §8.2
Sandbox-gated tests	10 reached	the containment family
Compiled `ogkm` oracles	**5 families**	VBIOS, GMMU-format, work-submit token, pushbuffer, USERD-chid — each compiles NVIDIA's *own* encoder as the oracle
TSan targets	4 files	nightly only
Fuzz targets	1	`parse_pushbuffer`; CI compiles it, does not run it
Implementation lines	120 979	`crates/*/src/**/*.rs`, excluding the detached generator
Test lines	118 160	`tests/`, `crates/*/tests/`, `fuzz/`

★ **"The suite runs in about 20 seconds" has been deleted rather than updated.** It was [unverified] when printed and the tree has quadrupled since; the honest replacement is that the authoritative run is no longer a stopwatch figure at all but a *phase table* (`scripts/run_full_suite.sh`), whose mutation phase alone ran 33 374 s on its last outing. The OS-free property the 20-second figure was really claiming — no GPU, no OS dependency, no wall clock — does survive, and is still what makes the adversarial suites affordable on every change.

7.2 The CI gates — and the gate that GitHub cannot be

There is one workflow file with six jobs, and the `stable` job grew from fourteen steps to **twenty-three**. Seven of the nine new ones are *reached-count* floors for oracle families that only exist on a box with the vendored NVIDIA trees, plus an *ABI-quarantine* gate, an *ogkm-version-tag* gate (every NVIDIA citation names its tree) and **the claim-ledger gate** described below.

The bridge-exclusivity gate fired on its first run, on `kayfabe-gsp`'s own stale doc paragraph claiming the bridge would live in `kayfabe-fwd`. That is worth more than the gate itself: a gate that has never been observed to fire is not evidence, and this one earned its keep before it was five minutes old.

★★★ **GitHub CI is opportunistic convenience. It is not the definition of green**, and the project says so in an owner ruling rather than leaving it implied: *"this project cannot rely on gh for iterating for free hardware … ensure whole suite can still be run if you have real gpu."* The authoritative run is `scripts/run_full_suite.sh` on real hardware, whose exit 0 means *everything this box can run, ran*.

The failure it is written against is this paper's own founding doctrine. A test that is COUNTED but never PASSES is worse than a missing test. There are two live instances and both are honest about it: the 51 `/dev/kvm` tests and the 10 VBIOS-oracle tests are counted on GitHub and never run there, because `ubuntu-latest` has neither the device nor the vendored trees. Both print a marker on the *skipping* arm, straight to stderr, deliberately bypassing libtest's capture — because capture swallows output on the **passing** path, which is the arm that matters. ★ And the runner does not know those families by name: it **derives** them from the markers a run emits, so on a box whose profile promised the capability a single `SKIPPED` is a red run. *A gate quantified over a hand-list is a smaller true statement.*

7.3 The claim ledger — a gate on the paper's own vocabulary

The most unusual instrument in the repository, and the one this document is itself held to. `scripts/claim_ledger.py` sweeps every tracked Rust source and design document, finds every prose *block* making an epistemic claim (*measured*, *observed*, *verified*, *confirmed*, *proven*), and asks one question of each: **does the block name what is behind the claim?** It classifies into [measured] / [inferred] / [assumed] / UNATTRIBUTED, and enforces three bars, currently **382 unattributed (a ceiling), 66 conflated (exact), 17 bare-hardware (exact)**.

The middle class is the named failure mode. CONFLATED means a block says *measured* or *proven* and the only thing behind it is a *citation into source* — `ogkm`, nvproxy, or the C artifact. *Reading the open kernel modules tells you what the driver does. Only a live boot with real work tells you what happens.* The bar is exact in both directions, because a gate that only catches growth leaves slack a later claim can be added into.

★ **Four things this gate says about itself, and they are the reason to trust it.** **(i) The universe is derived, never listed** — `git ls-files` plus untracked-but-not-ignored files, after a measured incident where a pre-commit run read 383 with three new modules unstaged and 385 the moment they were added. **(ii) It cannot check that the evidence exists**, that it says what the claim says, or that it was ever run: a block citing an imaginary capture passes. *Adjudicating a citation is human work* — and §3.1 is the worked example of a citation gate being satisfied by a source that said nothing. **(iii) The unattributed bar is a ceiling and is deliberately weaker than the other two**, "a worse gate honestly described, not a better one described optimistically", because an exact pin across a tree several agents write into at once turns an unrelated doc commit red — *which is how a ratchet decays into a formality.* **(iv) Two of its four firings were false positives and both moved the pattern, not the bar.** One site cited `ogkm` and then said, in the same block, *"No Ada card was measured for this number"* — the exact honesty the gate exists to encourage, scored as the defect it exists to catch. ⊘ *Absorbing that site into the bar would have been the cheaper edit and the wrong one: a gate that penalises the behaviour it is for teaches people to stop writing it down.*

This document was checked against that gate (§A), and the ledger stands at 382/66/17 with it. ⊘ *With one honest note*: `.tex` is not in the gate's universe — it sweeps `*.rs` and `*.md` — so **the whitepaper is the one artifact in this repository the claim ledger does not police**, which is worth knowing before trusting any *measured* in it more than the design document it came from.

Job	When	What it asserts
`stable`	every push + PR	**Twenty-three steps**: build; **test with KVM forced absent**; *six* reached-count floors (KVM, sandbox, VBIOS-oracle, GMMU-oracle, token-oracle, pushbuffer-oracle, USERD-chid-oracle); clippy `-D warnings`; `fmt --check`; the hexagonal-boundary grep; the VMM-vocabulary grep; **the generation-name grep** (no concrete chip or driver version in the logic crates); **the ABI-quarantine gate** (a C layout is quarantined, or provably own-wire); the `unsafe`-surface `ls` gate; the host-pointer gate; the GPA-accessor gate; the bridge-exclusivity gate; **the ogkm-version-tag gate** (every NVIDIA citation names its tree, ratcheted at 161 untagged); **the claim-ledger gate**; three unsafe-containment asserts including the **two-crate ratchet at 87 and 47**; and `fmt --check` for the fuzz workspace.
`aarch64`	every push + PR	`cargo check --workspace --target aarch64-unknown-linux-gnu`
`nightly-fuzz`	every push + PR	the fuzz harness *compiles*
`slow`	nightly	the full suite with `KAYFABE_SLOW=1`
`tsan`	nightly	ThreadSanitizer over four threaded targets, `-Zbuild-std` so std's own synchronisation is instrumented
`mutants`	weekly	`cargo mutants` over every production crate

7.4 Exercised versus merely present — the project's own core doctrine

This distinction is the most useful thing in the repository, and it is worth reproducing the incident that generated it, because it is the template for every criticism in §8.

> The conservation ledger's first census over the mean run reported **0 leaked objects** — and that was a **true negative, not a pass**. A design document claimed the workload "exercises T0 thousands of times per run"; the churn it actually drives adds and removes *RPC* bindings, which own nothing host-side, and the script's two real subset-frees targeted channels that phase 0 deliberately leaves virgin. **The path had never been reached.** Reaching it took a new test phase. Once reached, the honest baseline was **24 objects, 6 mappings and 24 576 GPA bytes leaking, linear in frees.** [docs/design/testing_doctrine.md §1]

Three rules follow, and the project applies them: a reclamation test that does not first allocate proves nothing; every instrument needs a non-vacuity assertion (a case where it is known to read non-zero); and **bite-check the instrument, not only the fix** — reverting the fix must move the number, and if it does not, the number is not measuring the fix.

Bite-checking is real practice rather than aspiration, and the evidence is that the repository records bite-checks that *did not bite*: at least six such notes are written into the source at the site they concern, each explaining what the neutering failed to move and what was added as a result. One has been promoted into a permanent test that plants a gap and a repeat in a trace and requires the order checker to fire. A commit message from this week reads, in part, *"the bite harness itself lied twice."*

The doctrine also produces two mechanisms directly. Slow tests are gated by `KAYFABE_SLOW`, and when unset they print `SKIPPED (slow): …` **straight to stderr**, bypassing libtest's capture; nothing in the suite is `#[ignore]`d, on the argument that `#[ignore]` is invisible in a summary and impossible to count. And the KVM gate prints on *both* arms — `KVM-GATE: RAN` or `KVM-GATE: SKIPPED` — with CI asserting a floor on the sum, because the reverse failure (all 36 gated tests silently vanishing while CI stays green) is equally silent.

7.5 The mean test, and composed versus isolated runs

`tests/tests/l1_mean.rs` is the largest single test file (35 tests) and is designated *the arbiter*. Its standing is: *pass = the design survived contact; fail = the doc changes, not the assert.* The rule it exists to enforce is an owner directive with four obligations — drive realistic multi-process × multi-thread × multi-GPU traffic end to end through the mocks and assert the observable end state; make it mean rather than happy-path (free mid-flight, race a re-declaration against a still-publishing generation, kill a worker partway, run both lock modes); wire every new milestone into it; and **never narrow a test to make it pass**.

The justification is measured, not asserted. One identity round landed 7 bite-checks; **none of the 363 pre-existing tests caught any of them**. Every real defect that day came from a composed run or a newly sharpened criterion. In the doctrine's words: *"the happy path found nothing, all day."*

A representative finding: a test named `worker_death_retires_the_proc_loudly_and_never_resurrects` was green — and green *only because it never issued an apply after the HUP*. The composed run put a worker hangup and an alloc/map-heavy workload in the same run and found that an out-of-band retire is undone by the next refresh: a guest able to crash its isolate worker got a **clean new isolate** on its next RM event. The first fix for the related leak class was a use-after-free of its own making.

7.6 The conservation ledger

The instrument all eight teardown paths are measured by. `HostLedger` replays the recorded verb log and demands every acquisition matched by exactly one release and nothing released that was never acquired — objects *and* mappings. A test may declare a `ResidueClaim` for what it knowingly leaves outstanding, and the counts are compared **for equality, not as a bound**: *less* residue than declared also fails, so a fix that closes a leak must delete the claim that excused it. There is deliberately no `allow_leaks` escape hatch.

Its stated limit, recorded in the source: the ledger increments only on *success*, so it says nothing about failed verbs. That is why a separate `VerbFailure{err, orphans}` type exists to carry handles out of a chain that died partway.

7.7 The mutation gate — five lies deep, and the fifth is the best one

The gate mutates the code and measures whether the suite notices. It has now been the most valuable instrument in the project *and* silently wrong **five** separate times, every one of them wrong *upward*. That asymmetry is not coincidence and it is the single most important thing to understand about this instrument: every way

an instrument like this fails makes the code look better than it is, because a mutant that is not run, not built or not counted is a mutant that never survives.

Lie 1 — it tested nothing. `kayfabe-fwd` and `kayfabe-rt` have no unit tests of their own; without `--test-workspace true`, cargo-mutants ran an empty suite and reported everything missed. (This one is the exception that proves the rule — it failed *downward*, and was therefore noticed immediately.)

Lie 2 — compiler crashes vanished from the denominator. cargo-mutants rewrites one file per mutant in a copied tree that reuses one incremental dep-graph cache, and that cache does not survive the churn — rustc ICEs. cargo-mutants cannot tell a compiler crash from a type error, so it files the mutant *unviable* and **silently removes it from the denominator**. Measured: **136 of 293 mutant builds ICE'd**, turning a real 88.95% into a reported 67%. CI now fails on any ICE in a mutant log, and on a zero viable count, *before* the threshold is read.

Lies 3 and 4 — the configuration file was never read, and the gate could report 100% from a broken suite — are covered in §8.6.

★★★ **Lie 5, measured 2026-08-03, and it is the sharpest of the five because the score was *exactly* 100%.** A full-suite run on a rented GPU box printed:

```
mutation score 100% (killed 1936 / viable 1936), threshold 91%  ok (33374s)
```

and, *in the same phase*, fifteen `No space left on device` errors plus `Error: write outcomes.json`. It had found **8 738** mutants and scored **1 936**. **6 802 mutants — 78% — were unaccounted for, with zero `MISSED` and zero `TIMEOUT` lines. They did not survive and get counted; they vanished.**

The mechanism was already understood *in that function*. The ICE guard directly above it says a dropped mutant *"removes it from the denominator and INFLATES the score"* — and guards only the ICE trigger. ENOSPC does the same and worse: it stops the result files being written. `missed.txt` was never created, the counting helper read its absence as **0**, and `viable = killed + 0` forces the score to exactly 100%.

★★★ **This is the project's own FIFTH ORACLE LIMIT reproduced inside its own harness** (§3.1): *an empty capture is evidence of nothing, not evidence of emptiness.* Absence read as zero, in a different repository, by a different mechanism, four days after the rule was written down. ⊘ **And a perfect score is the alarm, not the achievement**: 1 936/1 936 with no misses, after a run throwing ENOSPC, is the signature of a universe that collapsed under the campaign.

Four new refusals landed, each proven to fire, plus a control arm proving a genuinely clean campaign (8 738 found / 8 600 scored) still passes. ⚠ **And the harness was not wholly blind**: its summary already printed *"★ …and the disk is below the comfort bar"*. But the *phase verdict* contradicted the harness's own warning — **and the phase verdict is what a reader believes.**

The historical numbers, each true of the scope it was measured on: **99.2%** (245/247) on the pure L0 core; **92.44%** (159/172) on the L1 threaded surface. Both were measured over a population that *included the instruments*, because the exclusion file was inert.

Do not quote a mutation score from this project. There is still not a defensible one. The threshold in force is `91` (`MUTATION_THRESHOLD_PCT`), so the previous revision's "there is currently no mutation bar in force at all" is superseded — **but a bar in force is not a measurement, and the only recent campaign to complete is Lie 5's, which is void.** The project's own rule is that the bar is derived from a measured baseline under the configuration CI will actually use, and **no such run has completed**. Its number is not available to this document.

The campaign that *has* completed, labelled honestly. 2026-07-28, 2 898 mutants, 5 h 49 m: 2 305 caught, 112 missed, 3 timeout, 478 unviable. Product-only, with scaffolding removed from the denominator, that computes to **95.52%** (**95.38%** if a `TIMEOUT` is not counted as a kill, which is a real choice — cargo-mutants scores a timeout as *killed*). ⚠ **That run's tree was modified underneath it while it ran**, by a concurrently dispatched task, and the number is therefore reported as an indication and not as a measurement. It also does not describe the population CI will now test: with the configuration finally being read, `--list` yields 2 667 targets, not 2 898.

What that campaign found, which is worth more than its score. `kayfabe-gsp` is the floor at **92.71%** and holds 36 of the survivors — exactly as expected for a crate no campaign had ever covered, and the direct answer to the previous revision's "no mutation campaign has ever run over them". `kayfabe-abi`'s encode side, previously a named gap, is **closed**: 195 mutants, zero survivors. Of the 115 survivors, **8** are provably equivalent (each proven by its sibling mutant being caught at the same site), **22** are dead public API or `Debug` diagnostics with no callers anywhere, **17** are scaffolding, and the remaining **68 are real untested logic, individually named**. A debt that is enumerated to 68 named items is a different object from a percentage.

And one methodological result from closing some of them, which cuts against the usual reading of a

surviving mutant. `b612a91` closed twelve `kayfabe-gsp` survivors and found that one of them — a `+` → `*` in `RegionMap::load` — was not guarding correct code. It was *masking wrong* code: the function reported `pages.len() + i` as the index of a misaligned page-table entry while `pages.push` ran in the same loop, so the reported index was $N + 2i$ and correct only for the first entry of each table page. A guest-facing diagnostic naming the wrong entry of a hostile table. **"This mutant survived" is therefore not automatically "write a test"; it is sometimes "read the function".**

Three of the targeted survivors were *deliberately not closed*: `RegionMap::is_empty` and `page_size` have zero callers repo-wide, and writing tests for them would be exactly the score-gaming the instrument exists to detect. `is_empty` cannot simply be deleted (`clippy::len_without_is_empty` requires it once `len` exists) and deleting dead public API is an owner call. That distinction — kill what a real readback naturally asserts, refuse to invent a probe for what nothing calls — is the whole difference between a mutation score and a coverage number.

7.8 The OS-free configuration, and why it exists

`KAYFABE_NO_KVM=1` forces the KVM probe to report the device absent, so the pure logic core plus mocks — no OS layer at all — runs on purpose on any box. CI runs *this* as its default test configuration.

The reason it was made explicit is the doctrine applied to itself. GitHub runners have no `/dev/kvm`, so CI was already running the OS-free configuration — **by luck of what the runner image happens to lack**. It was unreproducible on any developer box, and it would have lapsed silently the day a runner image started shipping the device. In the commit's own words: *"a guarantee that holds by luck is the same shape as a green instrument on an unexercised path."*

The owner's question that prompted it is worth repeating because it is the right question: *"I hope you didn't delete the mocks so the core can still be tested without OS layer? and that must pass as well so we don't lock in architectures/os semantics/qemu and later regret."*

7.9 What the suite does not prove

- **It does not prove anything about NVIDIA.** Every test runs against `MockArch`, whose encodings are deliberately *not* NVIDIA's. That is the right design for proving the seam, and it means the suite cannot catch a wrong belief about the real protocol. Only the C artifact and the bench can do that.
- **The with-KVM configuration has no automated gate.** CI runs *only* the KVM-absent configuration. The 36 tests that use a real `/dev/kvm` — including the entire real memory plane and the vCPU/MMIO dispatch — run on developer boxes and the bench, and nowhere else. This is the largest hole in the CI story, and the workflow says so in its own text rather than hiding it.
- **The fuzzer is compiled, not run, by CI.** One target, over the one place raw guest bytes reach the core. Its two historical findings were minimised into deterministic in-tree regressions, which is the stronger form. The corpus is git-ignored, so a fresh clone starts from empty.
- **Single-threaded logic testing of a concurrent system.** The core is tested deterministically because it is a pure state machine; the concurrency is tested separately by the threaded suites plus TSan. The argument is sound, but it depends on the core genuinely having no interior mutability — which is asserted at compile time, and is the sort of property worth re-checking if you are looking for holes.
- **The page-size axis is designed and not built.** The testing doctrine cites `[4 KiB, 16 KiB, 64 KiB]` as an application of its "both modes ship from day one" rule. The environment variable that would select it does not exist in any `.rs` file — only in forward-looking comments. The `LockMode` half of that same rule *is* built and exercised.

8 Discussion: the good, the bad, and the risky

8.1 What is genuinely good

The purity constraint is real and it is mechanised. A "pure core" claim is usually aspirational. Here it is three CI greps, a manifest-level `forbid`, a naming rule that makes `ls` the audit tool, and a ratchet on the exact number of `unsafe` relaxations. The external dependency surface is three crates, two of them dev-only.

The failure ledger is unusually honest and it is load-bearing. Two design documents carry "contact logs" — `l1_concurrency.md` §12 and `l1_os_shell.md` §14 — recording every place building the code proved the design wrong. They are long. They contain entries whose titles are things like "the criterion was wrong", "boundary-1 broken", "two sections of §3 contradict each other", "the design was wrong about its own severest claim". A project that writes these down is a project whose other claims are worth more.

The adversarial suites found real bugs before hardware existed. 15 at L0, more since at L1, including two

control-plane *wedge* bugs reachable by a hostile guest process — a parked `SET_PAGE_DIRECTORY` and a parked `MAP_MEMORY_DMA`, each of which could be aimed via a dup alias to make a benign third party's allocations fail forever. Both were found by a property test, not by inspection. *The same parked-fact machinery produced this revision's worst core defect too* (§8.5), which is a fair reading of where the difficulty in this design actually lives.

The bridge was designed on paper before it was built, and the design was then reported against. `gsp_core_bridge.md` was written first, in full, including the finding that changed the roadmap; the four implementation commits then recorded roughly two dozen places where that design met the compiler and lost, *by section number*, including three of its own test obligations turning out to be unbuildable and one turning out to be wrong. A design document that is graded by the code that implements it is worth more than one that is merely followed.

Both lock modes and both fallbacks ship as first-class tested modes. The argument is sharp: the slow path is the path the system takes *when something has already gone wrong*, and a fallback exercised only by the fast path declining has test coverage proportional to how often the fast path fails — which is the quantity the optimisation exists to drive to zero.

The ABI oracle design. Pinning generated layouts against three independent readings, with a test that documents exactly how they disagree, is better than what most projects do, and it caught the C artifact's own parity test being weaker than the code it guarded.

★★ **Compiling NVIDIA's own code as the oracle, five times.** The strongest testing idea in the project is newer than any of the above and does not appear in its doctrine document. Where a wire encoding had to be settled — the doorbell token, the GMMU page-table format, the pushbuffer codec, the USERD channel-id decode, the VBIOS layout — the answer was not transcribed from the headers and asserted. **NVIDIA's own generator, reader and recombination logic were compiled and run as the oracle**, and our implementation differentially tested against them. A transcription cannot detect a shared misreading; a compiled oracle can, and each of these caught one. Five such families now have their own CI reached-count floor, so a box that cannot run them says so instead of quietly counting them as passes.

★ **Corrections are struck, not deleted — and the practice survived contact with being wrong in public.** `boot_measured_2026_08_01.md` carries sections marked **REFUTED** at their heads, kept in full, with the observation preserved and only the inference withdrawn: *"it is kept, not deleted, because the observation below is accurate and only the inference was wrong — and because this is the clearest example in the file of the failure mode the whole document exists to resist: a chain of assertions read as a chain of causes."* A project that writes that down about itself is a project whose other claims are worth more.

8.2 ★★★ The execution plane is the wall, and it cannot be faked

This is the section that replaces every previous revision's headline, and its shape is different from theirs. The last three said, in various words, *"the critical path is one layer up."* It never was. **The critical path was always the plane where a real engine has to move real bytes, and no amount of protocol work reaches it.**

How the wall was found, and why it is not a control

The boot climbed. Rung after rung, a control was served, the guest advanced, and a new control was demanded — the ordinary shape of this work, and the record of it runs to seventeen hundred lines of dated, revision-stamped boot log. Then it stopped being that shape.

The guest dies in the memory manager's scrubber, constructing `CeUtils`. [inferred] from source and immediately past that point, `memmgrInitCeUtils_IMPL` calls `memmgrTestCeUtils` **unconditionally**: it writes `0xAABBCCDD` to framebuffer, issues `memmgrMemCopy` with `TRANSFER_FLAGS_PREFER_CE`, reads it back, and asserts `sysmemData == vidmemData`. **A functional end-to-end test of the copy engine with a read-back comparison. There is no control-shaped answer to it.** Its status returns through `NV_ASSERT_OK_OR_RETURN` into a `kernel_fifo.c` arm that is `NV_ASSERT(0)` — **fatal for any non-`NV_OK` in this phase**, so the pre-init sweep's "`NV_ERR_NOT_SUPPORTED` = amputate and carry on" rule does not apply here.

Every escape hatch is closed, by citation, and one of them is closed in an unusual way. The Blackwell-only property that skips the test takes the `else` arm on GA106; the `bDisableGlobalCeUtils` regkey requires modifying the guest, which is outside "stock, unpatched"; the `IS_VIRTUAL_WITHOUT_SRIOV` arm makes the interrupt self-test permanently unpassable; the scrubber's own gate is cleared only by build configurations we do not have.

★★★ And the obvious lever, `!IS_SILICON`, **does not exist**. `IS_SILICON` is `!(IS_EMULATION || IS_SIMULATION)`; the

three flags behind it are **declared in the driver's own headers and assigned nowhere in the tree**, and the emulation property is never set either. It is structurally constant true. *This is not a lie we decline to tell; it is a lie we cannot tell.*

⊘ **And the C artifact does not rescue us.** Its per-control synthesis ladder is for *incomplete captures*. The C reached this same point and needed a **real CE**.

⊘ **The honest limit, and this paper marks it because the design record does. No boot has ever reached** `memmgrTestCeUtils` — the deepest one died at `objCreate(CeUtils)`, two statements short. That specific wall is [inferred] from source, never [measured]. What *is* measured is that the driver demands a scheduled channel with an armed completion and a work-submit token, and that serving those individually walks the boot from each to the next without changing kind.

E0–E7: what was built, and the two things it establishes

Eight increments, each with a named run at a named revision on named hardware.

Increment	What it built, and what a boot then measured
E0	The isolate plane becomes selectable at runtime. ★ The boot **falsified** the claim it was built on: the spawn was `realize`-time, ∼28 s *before* the guest opened the device.
E0b	Spawn made lazy, so it follows the guest's action. Re-measured, not assumed: child at `t+33` s against a device open at `t+31` s, and a fault-injected arm goes red.
E1	An isolate can *refuse* and say why; a census the device prints on teardown.
E2	A guest MMIO write reaches `SharedDevice::doorbell`. ⊘ And the same run measured that **the guest driver rings no doorbell at this wall** — 0 arrived across boot, `modprobe` and device open.
E3	The doorbell token decode, settled by compiling NVIDIA's *own* generator as an oracle, then paired against six real tokens on hardware. ⊘ The runlist field could not be reached: the control that would give it is `KERNEL_PRIVILEGED` and refuses *root*.
E4	The GA10x USERD model and pushbuffer codec. ★ It **refuted its own seam**: a real CE copy is *five* method runs and the launch method carries none of its operands, so a stateless per-method decoder is structurally incapable. ⊘ No boot — every number from NVIDIA's macros on a build box.
E5 ◐	Pushbuffer reads now *translate* through the address table. **PARTIAL**, and asserted so by a test that goes red the day it stops being partial: the CE page-table-write populate source witnesses a root page and no further.
E6	**The join.** `CeEvidence::copied() == true` on real hardware.
E7	The R1 spawn deferral (§8.12) — the real isolate plane can be switched on at all.

★★ What E6 establishes, stated as narrowly as the evidence allows. On an RTX 3060 (GA106) with host driver 580.159.04 open, twice at two revisions with byte-identical evidence: a real GA10x GPFIFO entry naming five real CE method runs went through `read_pushbuffer` → `decode_run` → `partition_ce` → `plan_ce` → `Worker::execute` → a real host copy engine. 4 096 bytes moved, the release semaphore landed, and the destination read back correct *through an independent second mapping* — first word and last. **The predicate was watched to fail first**, on the same hardware, with a live mutation: setting the copy's source to its destination left the engine executing a copy that *moved nothing*, and the evidence said so.

⊘ **And what it does not establish, which the test says in its own header. (1) No live guest drove it** — it is a guest's *bytes* and a guest's *declarations*, not a booted guest's submission, because a booted guest dies before it ever rings. **(2) Nothing about the sandboxed isolate** — the acceptance runs an *in-process* backend, because a CPU mapping and a witness both die at the process boundary. **(3) Nothing about guest RAM** — the method words live in a mock; no hypervisor is running. **(4) Nothing about the completion plane** — nothing is rung for the guest and no interrupt is raised.

★ **And it needed two arms because a published backing is CPU-opaque by design.** The first draft died at its own sentinel write: memory minted for the GPU carries a no-map flag, so `NV_ESC_RM_MAP_MEMORY` refuses it — the sentinel cannot be written and the answer cannot be read. Arm 1 (published at the guest's own VAs) proves the *address* plane; arm 2 (hand-mapped, device-local) proves the *bytes*. **Neither substitutes for the other**, and relaxing the flag to make the diagnostic work would have been changing the product to fit its instrument.

★★★ The control found a hang, and it is the better finding. E6's own control arm — "the same boot with the real plane off must *not* produce this" — turned up a defect that the acceptance could not have.

`Isolate::checkout` answers `None` for two conditions its own documentation runs together: *the pool is saturated*, and *the isolate is retired and refuses new checkouts*. The forwarding layer passed both up identically, and the device treats that as *backpressure*: drop the locks, park on the pool gate, re-enter. The shipped default plane is a stillborn isolate with pool width **zero**, retired at birth — so the generation never moves and **the vCPU thread parks forever**. It was unreachable until E6 only because nothing had ever routed as far as a checkout, and **the configuration that hangs is the shipped default**.

And the first fix was wrong in the way this project keeps being bitten. It changed one function. Every unit test went green — and the test named `a_permanently_dead_isolate_is_REFUSED_and_does_not_park_forever` still *timed out*, because the threaded path does not go through that function. *A mutation must be shown to change behaviour on the path the product takes, not merely to change bytes.*

8.3 ★★ A trace of a boot that SUCCEEDS — and the ladder, enumerated

⚠ **Scope note, first, because it is a claim about where code lives.** The recorder and the demand lists described here are on branch `replay-conformance`; they are **not on** `master`, which is where this document lives. Everything below is real, committed and dated, and a reader checking out `master` will not find the paths. This paper would rather say that than either omit the work or cite a path that resolves to nothing.

Until now every trace this project owned was of a boot that **fails** — the C's `cap1` ends where the C's emulator stopped, and our own boots end where ours does. A trace of a failing boot has an ambiguity that cannot be argued away: **every absent control is indistinguishable between "the driver never wants it" and "the driver died before asking."**

A recorder now exists inside CPU-RM itself. Two capture points in NVIDIA's own `message_queue_cpu.c` — one per direction — recording the *complete* message-queue element, header and body, into a `vmalloc` ring drained through `/proc`. **Two call sites. That is the whole instrumentation surface.** Its one invariant is stated as such: *a record has its bytes, or it does not exist* — fill-and-stop, never overwrite, and a non-zero drop count makes the trace unreplayable and the decoder rejects it.

What it captured, 2026-08-03, on a real GA106: 1 076 records (535 out, 541 in), 1.17 MiB of a 64 MiB ring, **zero dropped**, 14 distinct RPC functions, 620 control elements resolving to 310 request/reply pairs and **104 distinct controls** — and, the line that matters most, **replies declaring params with no bytes present: 0**. Two independent GSP bring-ups in the trace (sequence numbers restart), with *zero* retransmits inside either, which makes it a repeatability check as well as a capture. **The boot succeeds**: `nvidia-smi` reports the card, and the capture script fails if it does not.

★ **Six of the C oracle's empty rows now have bodies**, including the one that cost four rungs: `0x20802a08` answers 20 480. *A genuinely zero-length reply is now distinguishable from an unmeasured one* — which is the whole of §3.1's fifth limit, repaired at the source rather than worked around.

The ladder, enumerated rather than discovered one boot at a time. A successful boot demands **104** distinct controls. The failing `cap1` capture reaches **53**. **53 are demanded by the successful boot and never asked for by** `cap1`, and two go the other way. ⚠ *The two 53s are a coincidence of this capture and the source doc goes out of its way to deny a relationship between them*; this paper repeats the denial. And the ladder's *serve* obligation is at most **93**, not 104, for the reason in the next paragraph.

Each control is bucketed **DATA** (the reply is the whole answer; servable from a table), **ACT** (params all `[in]`; the reply only acknowledges something that *happened*), **MIXED**, or **UNKNOWN** — statically, out of the driver source, with a citation to both the params struct and the code that decides direction. **81 of 106 (76.4%) are bucketed with a citation**; 79 are DATA or ACT; **25 are UNKNOWN**. ⊘ *The classification is a reading, not a measurement*, and the docs keep the two in different sentences.

★ **The 25 unknowns are not a tail — they are one wall.** Three are a binary-API class with no export, no flags and no struct on the CPU side. The other 22 are GSS-legacy commands that RM forwards verbatim to GSP with a comment saying, in NVIDIA's words, that it no longer supports them and is forwarding them so GSP can decide. **None of the 22 appears in either vendored driver tree**; the versions do not disagree, they are both silent. *The ladder is not 53 hard problems; it is 31 easy ones and one closed-firmware wall.*

★★★ **And the finding that bears hardest on our own design: a real GSP refuses 13 controls on the boot that works.** Eleven come back `NV_ERR_NOT_SUPPORTED` unconditionally; two are conditional. `NV_ERR_NOT_SUPPORTED` **is** `0x56` **— the same status this port emits when nobody claims a command.**

And CPU-RM anticipates 7 of the 7 it can name. Six are on a list RM keeps of commands whose unsupported

status it declines even to *log*; the seventh has an explicit tolerance arm at its call site that converts the refusal into a normal branch. The four it does not anticipate are exactly the four GSS-legacy ones, **which the source cannot name**. ⊘ Scope, honestly: the list RM keeps is written about a vGPU guest, so it evidences that RM has a named class of legitimately-unsupported controls containing these ids — not by itself a statement about the bare-metal path. **What makes the pairing strong is that the refusal was [measured] on a real GA106 while the anticipation was [inferred] from source. Two instruments, one answer, neither doing the other's job.**

⇒ **Refusal is ordinary protocol behaviour, not merely our posture.** That is a real strengthening of this project's refusal-first design: for eleven controls, refusing is demonstrably what the real device does on a boot that then works.

⊘ **But refusal is not a free pass, and the correction here is worth more than the rule.** An earlier document concluded from the two conditional refusals that *"a static policy cannot express it."* **That was retracted as too strong.** A policy keyed on the *command id* cannot; a policy keyed on the *params* expresses one of them exactly — the status is a deterministic function of the argument, re-measured identically across both independent bring-ups in the trace — and is the only shape that could ever express the other. **The surviving distinction is cmd-keyed versus params-keyed, not static versus dynamic.** One row cannot be served in any form: two identical requests return different live PCIe byte counters.

And two facts a replay needs are independent. A control can be **DATA and still be refused by hardware**: two controls bucketed DATA off a clean reading of the driver get `NV_ERR_NOT_SUPPORTED` from a real GA106 on a boot that then works. *The bucket says what the reply means; it never said the reply exists.*

What the recorder cannot see, stated by its own author. It answers *what a well-behaved boot demands*, and nothing else. It is a strong *positive* oracle and a weak negative one — it says nothing about what the driver does when refused, when something arrives out of order, or late, so our refusal-first cases must be *authored*, not recorded. It covers one part, one kernel, one driver, and only the open driver — the closed one cannot be instrumented and the correspondence is [assumed]. Nothing in `crates/` reads the format. And ⊘ **observational neutrality is not proven**: the hooks add a `memcpy` of up to 64 KiB under a spinlock on the RPC path, the 88/88 cross-validation is against a *differently instrumented* build of the same driver, and **no measurement in that task establishes what an uninstrumented driver issues**. ⚠ For the same reason this paper does not call it a stock-driver capture: it is real GA106 silicon and real GSP firmware, with the CPU-side driver rebuilt from stock *source* plus two hooks.

8.4 ★ The bridge exists — and it does *not* unlock compute

This section replaces the previous revision's headline, which was: "the critical path is the bridge, not the crate." The bridge is built. It was designed first (`docs/design/gsp_core_bridge.md`, 865 lines) and then built in four stages, each independently green, each recording where the design met the compiler. If you only read one thing about it, read §2.7 of that design document, which is the finding that changed the roadmap.

What was built

`translate(&DriverAbiTable, &RpcCommand)` is a stateless free function; `GraphPolicy` is the `CommandPolicy` the boot FSM calls, doing translate → `Gpu::apply` → reply; `kayfabe-abi` gained the per-class alloc table and four params decoders; and the control arm (function 76) decodes what it can model and **refuses the rest by name**. The refusal set is fifteen variants wide and every one of them carries the number that caused it, because the census is the stage's real output: it says which decoder is missing, not merely that one is.

★★ `MAP_MEMORY_DMA` is not on the wire, so the bridge cannot deliver compute

This is the finding to carry away, and it is a roadmap fact rather than a code fact.

On a GSP-client GA106, `rpcMapMemoryDma` (function 14) and `rpcUnmapMemoryDma` (function 15) are `_STUB` **through the entire HAL chain** — `_GA106` → `_GA102` → `_GA100` — and the stub returns `RPC_UNKNOWN_FUNCTION`. The real implementation is installed only by the *vGPU* IP-version table. **So** `RmEvent::MapMemoryDma` **has no producer on this path, and no stage of this bridge builds an** `NVOS46` **decoder.**

Three independent oracles agree, which is why this is stated flatly: the open kernel modules' own generated HAL tables; the C artifact, which booted a real GA106 guest to CUDA and has **no function-14 hook at all**

in its complete snoop set (47/76/21/103/10/65/70/72/73); and two C-era design documents that say it in words — *"there is no per-map GSP-RPC carrying VA↔phys"*.

The consequence, stated so nobody reads the bridge as one stage from compute. The address table's populate sources are the `GPU_PROMOTE_CTX` control and **CE page-table-write capture** — and both are `kayfabe-fwd` / `kayfabe-mmu` work in different crates, neither of which is built. `GPU_PROMOTE_CTX` is not even shaped like an `RmEvent`: it declares a *set* of `(gpuPhysAddr, gpuVirtAddr, size, bufferId)` entries, which is an address-table population rather than an object-model edge, so it does not belong in this seam and was deliberately left out of it. Finishing the bridge to B6 would give a complete object model from wire bytes and **still** leave the address plane empty. The compute path is not one stage away; it is a different plane.

★ `SET_PAGE_DIRECTORY` is not how an ordinary address space declares its root

B4 was built to decode the control that carries a page-directory base. It settled the question *against* the design that asked it.

On a bare-metal GSP client, `SET_PAGE_DIRECTORY` reaches the wire **only** for an address space marked `SHARED_MANAGEMENT` or `IS_EXTERNALLY_OWNED` — `gvaspaceExternalRootDirCommit_IMPL` asserts on exactly that, and its only callers are the UVM / gpu-ops path. Every *ordinary* RM-managed VAS declares its root through `NV90F1_CTRL_CMD_VASPACE_COPY_SERVER_RESERVED_PDES` (`0x90f10106`) at construct time, as `levels[0].physAddress`, on a path that is on by default for any GSP client. There is a third: `NV2080_CTRL_CMD_INTERNAL_GMMU_COPY_RESERVED_SPLIT_GVASPACE_PDES_TO_SERVER`.

The design's arm was therefore **necessary and not sufficient**. All three commands are refused as a *distinct* variant, `PageDirControlNotModelled{cmd}`, rather than folded into the generic unknown-control refusal — and the reason is the sharpest small argument in the bridge: **a dropped page-directory declaration is invisible**. The VAS never routes, every channel defers at its first doorbell forever, and nothing anywhere says why. A separate refusal variant converts a silent hang into a counted, named record of exactly which decoder the address plane is waiting on.

Two smaller results from the same arm are worth keeping because they are the kind of thing a version table does not catch. `hVASpace == 0` is **not** "unspecified": NVIDIA's own header says verbatim, in both vendored trees, that zero means *the implicit VA space associated with the client/device pair* — a real object the RPC does not name and the graph has no node for, so passing it through would park a PDB on a node the guest never declared. It is refused. And note the deliberate asymmetry: on the *alloc* arm, a zero handle means "nothing declared" and maps to `None`. Same zero, opposite meaning, documented differently by NVIDIA — so neither may be inferred from the other.

Where the design met the compiler

Roughly two dozen items are recorded across the four commit messages, and the pattern in them is more useful than any single one: the design's *taxonomy* claims survived and its *buildability* claims did not. "Known-and-inert versus unknown, there is no third state" was contradicted by the first stage's own row — there are at least four states. Three separate non-vacuity arms the test plan specified turned out to be unbuildable at the stage that specified them, one of them because it was also *wrong*: refusing a second, different PDB for one VASpace requires remembering the first, which a stateless bridge cannot do and which `RmGraph` had already decided the other way, with an argument (re-binding is protocol-legal; refusing would hang a conforming guest).

One item is a finding the design does not have and could not have had: **functions 72 and 73 never reach the policy at all.** The FSM returns on a no-reply disposition *before* calling `respond`, so those arms are unreachable through this adapter. Harmless today, because neither carries object-model content — and load-bearing to know, because a future no-reply function that *did* carry a fact would be dropped silently.

8.5 ★★ A live-memory-free defect — and the independent oracle had the identical bug

This is the most serious defect the project has found in its own core, and the part that matters is not the defect.

The defect, one line wide. `RmGraph::apply` drained its three parked-fact tables from the `Alloc` arm only, on the reading that a parked fact waits for its target's *allocation*. It waits for its target's **handle** — and `Alloc` and `Dup` are the two events that mint one. So a dup-of-a-dup whose middle handle was minted by a `Dup` stayed parked *forever* while being fully resolvable.

The measured consequence. `project()` follows parked dups through `origin_of`, so the process grouping saw the alias; the refcount never did. The undercount takes references to zero, the resource is destroyed, the component vanishes, the process is vacated — and **the reaper emits `Free` for memory a live kernel alias still references.** That is the project's own "never free host memory RM says is live" rule, reached through a door nobody was watching. It was three defects, not one: the same missing drain also left `Resource::pdb` unset for a parked page-directory declaration and never installed a parked mapping, giving a fault on a *legal* guest.

> ★★ **And the independent oracle carried the identical bug.** The property suite's `RefTracker` — the deliberately separate reference-count model whose entire purpose is to disagree with the implementation — also promoted parked facts only in its `Alloc` arm. Model and implementation were wrong *the same way*, so the property stayed green over a three-defect bug for as long as both existed. The correction is proven to be a strengthening rather than an assertion: with the model fixed and the implementation's drain removed again, the property fails on its own.

Why this is the most instructive entry in the paper. An independent oracle is this project's strongest single testing idea — it is what `kayfabe-abi`'s three transcriptions are, what `gspworld` is, and what the bridge's hand-hex fixtures are. This is the failure mode of that idea, in its purest form: *independently written* is not *independently reasoned*. The same author, holding the same wrong sentence about what a parked fact waits for, wrote it into both. Nothing about the shape of the test caught it; what caught it was a different invariant, derived from the other side of the graph, disagreeing.

8.6 ★★ The quality instruments were wrong — three of them, in one day

This is the real theme of the work since the last pin. The code got better; what got *much* better is the confidence one is entitled to have in the instruments that say so. Each of the three below was a live, silent, one-directional error in the machinery this document quotes.

1. The mutation configuration was never read. `mutants.toml` sat at the workspace root. cargo-mutants reads `.cargo/mutants.toml`. **The file was never opened, and every exclusion argued inside it was inert** — including the exclusion of `kayfabe-mocks` and `tests/src`, which is the one this paper has repeatedly relied on when saying "excluding scaffolding".

It was proven rather than inferred, which is the right standard for a claim of this shape: appending a bogus key to the root file changed nothing and produced no parse error; appending the same key to `.cargo/mutants.toml` fails loudly. `cargo mutants --list` goes from **3 760** targets to **2 667**, and scaffolding mutants listed goes from **361** to **0**.

The consequence for every number this document has ever quoted. Every mutation figure recorded before 2026-07-28 was measured over a population that *included the instruments*, and the "≈94.9% excluding scaffolding" figure — which this paper printed at the previous revision — was a **hand computation**, not something the tool ever did. That is exactly why it read as a parenthetical rather than as the gate: it was never the gate.

2. The gate could report 100% from a broken suite. cargo-mutants baselines with `--package=` naming only the mutated packages, while the mutants themselves run `--workspace`. Measured: the baseline runs about 200 tests across 39 binaries and **omits `kayfabe-tests` entirely** — the conformance suite that catches most mutants. A red conformance suite would therefore pass the baseline and then make *every* mutant look caught. Fixed with `needs: stable`, which makes the real suite a precondition of the gate rather than an assumption of it.

The same job's `--timeout 240` sat at or below honest completion (the workspace suite alone is 201 s on a 25-core box, and a 4-vCPU runner at `-j 2` is slower). **cargo-mutants scores a `TIMEOUT` as *killed***, so mass timeouts would have reported near-100% and the gate would never have fired. Raised to an absolute 1 800 s — and deliberately *not* to a `--timeout-multiplier`, because the multiplier is computed off the same baseline that omits the suite, and would reintroduce the identical inflation through a different door.

3. Thirteen test park-points hung forever instead of failing, producing zero output. The shape is a `thread::scope` whose cleanup is reached only by statements *after* the assertions: any failed assert unwinds past them, and the scope blocks forever joining threads that were never told to stop.

> **It had already happened, for twenty-three hours.** Three wedged test binaries were found alive on the box, three threads each, all parked in `futex_do_wait`, at ∼83 000 s. And the failure mode is worse than "buried": under CI-

realistic output capture **the panic message is never emitted at all** — the log stops at `running 1 test` and stays empty forever. One site hung *spinning*, burning a full core, and outlived the `cargo` process that spawned it.

Every one of the thirteen was **proven vulnerable before and proven fixed after by inducing a real failure at each site**, not by reading the code — which is the same bite-check discipline the project applies to its assertions, applied to its harness. The fixes are `Drop` guards armed as the first statement inside each scope closure, plus a barrier type that panics instead of blocking when its rendezvous breaks. Each guard carries an in-comment R1 argument naming its callee, because *a* `Drop` *is a call site* and R1 does not guard `Drop`.

And a fourth, smaller, which is the same species. The test count itself was wrong: 551 came from **stale prebuilt binaries** in `target/`. Two test files had 16 and 32 `#[test]`s at HEAD while the inherited cache ran 13 and 24. A related trap bit twice in one day inside the bite harnesses: `cp -a` and `shutil.move` both preserve mtimes, so a restored file keeps the previous mutation's compiled artifact and a whole bite table comes back *inflated*. Any harness that restores files must `touch` them.

What these four have in common, and why they belong in a whitepaper rather than a changelog. Every one of them made the project look *better* than it was, and every one was invisible from the output. An inert config, an omitted baseline, a timeout scored as a kill, a hang that prints nothing, and a stale artifact are all the same failure: **the instrument reported a number that no longer described a measurement**. This is the project's own founding doctrine — *"a green instrument on an unexercised path is worse than no instrument"* — turned around and pointed at the instruments themselves, and it is the strongest available evidence that the doctrine is being applied rather than quoted.

8.7 `kayfabe-gsp`: boot is modelled, reboot is not, and one open item has no proposed resolution

Two revisions ago this section said "34 lines, and it is the critical path". One revision ago it said "3 550 lines, and nothing calls it". Both are now superseded: the crate is 3 828 lines across 8 modules, it has a production consumer (§8.4), and the one invariant this paper flagged as prose-only — GSP-S1 — has been paid off in code. **What has not changed is everything below.** Three of the plan's nine stages are unbuilt and all three are the ones that need a GPU; one open item still has no proposed resolution; and the plan the crate was built from was written against the wrong driver version, a fact that keeps producing new consequences rather than being absorbed once.

What is built, and what "built" is worth here

The five seams the crate claims — GPU architecture, driver version, hypervisor, GPU lifetime, CPU architecture — each have a test that pushes on them, which is the difference between a seam and a decoration. Two are worth singling out because they are cheap to fake and were not faked. The lifetime seam is exercised by running *repeated* driver lives in one process: the C artifact's measured second-life failure (`msgqRxLink` timeout, `-7`, 71 064 retries) has exactly one cause in the driver's own source, `rx.size != size`, and the FSM is shaped so that cause cannot arise. The architecture seam is exercised by driving the same FSM through *two* deliberately-different register models, which is the same argument as `MockArch` one layer down.

The oracle situation has changed completely since the previous revision, and it is the one unambiguously good piece of news in this section. That revision said: *"Those traces do not exist: the capture patch is ∼60 lines against a C file this work does not own, and it has not landed."* **It landed.** Five captures exist, two are committed into this repository decompressed, and `kayfabe-crec` replays them against the real FSM in the default suite. ⊘ **And no replay has been closed** — see §3.1: `cap1` stopped on a missing observation at transaction 978, `cap1b` carries it to 1028, and what stops it there is a plane the C's recorder cannot witness at all. *"Byte-equivalent with the C" is therefore established up to a stated transaction and not beyond it*, which is a far better position than untested and is not the same as tested.

Boot is modelled. Reboot and resume are not.

The plan has nine stages. S0–S5 — the six that need no GPU — are the boot and steady-state protocol, and they are built. **S6, S7 and S8 are the three that need a GPU, and none is built.** The bench-offline blocker that also held S6 is gone (§8.15), so what remains is work rather than an obstacle — but it has not been done, and the two open questions behind it (**O3**, whether sequence numbers reset across a true `rmmod/insmod`; **O5**, which

boot mode a post-teardown re-acquire uses) are still answerable *only* by a bench run. This matters more than the stage count suggests, because the owner's standing requirement is that a GPU restart — idle, released, re-acquired — must work *with no bolt-on*, and restart is exactly the region S6–S8 covers. **And a boot that dies in** `RmInitAdapter` **never reaches a teardown**, so nothing on the bench exercises this region even incidentally. Boot is the part that could be modelled offline, so boot is the part that got modelled.

★ The sharpest open item: at 580 the resume handoff is an RPC *we* would have to send

This is §11-O7a of the port plan, and it is the one item in the whole component with **no proposed resolution**, because the evidence that would settle it is not in any open-source tree.

The mechanism is present at both driver versions and reads identically: a core resume resets into RISC-V, reprograms the LibOS boot-args address, starts SEC2 and waits on `NV_PGC6_BSI_SECURE_SCRATCH_14._BOOT_STAGE_3_HANDOFF`. What differs is **who initiates it**:

	580.159.04 (the bench)	**610.43.02 (the vendored spec)**
where the wait lives	inside the CPU sequencer executor, as opcode `CORE_RESUME`	promoted to a first-class HAL, `kgspExecuteCoreResume_TU102`
how it is reached	**only** through `_kgspRpcRunCpuSequencer` — i.e. only if *we send* a `GSP_RUN_CPU_SEQUENCER` event	locally, from two call sites in the driver. **Nothing is required of us.**

Three things follow, and none of them is comfortable.

It is not a layout difference, so a version table does not absorb it. Every other 580-vs-610 divergence the project has found is an offset, a field count or a constant — one row in a version-keyed table. This one is a difference in *who initiates*, which means a faked GSP that must emit sequencer buffers at 580 and must not at 610 has version-conditional *behaviour*. `docs/design/four_axes_of_variation.md` §5 rule 1 forbids exactly that: *"No* `if guest_version == …` *in a logic crate. Transitions fire on observed protocol facts, never on driver identity."* The two honest options named in the correction brief are a version-keyed *capability* on the FSM or a loud refusal, and the brief declines to choose between them on the grounds that choosing requires an answer it does not have.

The answer cannot be read out of the open source. What the open tree shows is that the path exists and how it is triggered. What it cannot show is whether the firmware ever triggers it on a path we must support, because the *contents* of a sequencer buffer come from GSP-RM firmware, which is not open. Specifying an emitter now would be guessing, and the plan says so rather than shipping a guess.

So the requirement is genuinely at risk, and this is the cleanest statement of it in the project. If any restart scenario in scope reaches `CORE_RESUME` at 580, then a faked GSP that never emits `GSP_RUN_CPU_SEQUENCER` cannot drive it, and "GPU restart must work with no bolt-on" is not currently satisfiable by the design as written. *Whether* such a scenario exists is unknown [unverified]. The next step is a reading task, not a hardware task — enumerate the callers of consequence of the sequencer executor at 580 — and it is recorded as O7a rather than resolved.

One further sharpening worth carrying, because it cuts against the project's own instinct: the relayed finding that started this said "580 has a resume handoff 610 never reads". **That half was wrong** — 610 has the identical routine reading the identical register. The correction log reports it as wrong rather than quietly fixing it, which is the behaviour a reviewer should be checking for.

★ The plan was written against the wrong driver, and one item was RESOLVED backwards

This is the most instructive thing that happened to this component, and it is not flattering.

The port plan took its NVIDIA citations from the vendored open kernel modules at **610.43.02**. **The bench runs 580.159.04.** The two do not merely differ in offsets; they differ behaviourally, and 3 550 lines were built against the un-caveated plan before anyone vendored the matching tree. Eight claims changed when `ogkm-580.159.04` was finally vendored (`7af23cb`): the element count is *read from a field* at 580 and *derived from a length* at 610; MCTP/NVDM transport headers do not exist at all on the 580 GSP path; the layout break interval narrowed from `(570, 610]` to `(595.84, 610.43.02]`; the init RPCs are queued *before* bootstrap at 580 rather than inside it; the suspend sentinel is an exact equality at 580 and a mask at 610; and the queue geometry is compile-time at 580 rather than negotiated, which removes the "the guest declares its own geometry" argument the plan had presented as its highest-leverage design choice.

★★ And at this revision the same seam bit inside a range the project claims to support, which is a strictly worse shape than "the plan cited the wrong tree". `NV_CHANNEL_ALLOC_PARAMS` **diverges at offset +32**: 610 inserts `hHandleVASpace` there, where 580 — *the bench driver* — has `hUserdMemory[0]`. A generated 610 mirror would mis-read **every field from +32 onward** for the guest we actually run, `engineType` included, and would do so *plausibly* rather than loudly.

The response is the one the four-axes rule implies and it costs something: **the struct is deliberately not generated at all**, only the 32-byte agreed prefix is decoded, and a test named `the_channel_alloc_prefix_stops_where_the_two_trees_stop_agreeing` asserts the *absence* of the generated struct so that nobody helpfully adds it later. What that costs is concrete: `engineType` lives past the prefix, and **a CUDA process's GR and CE channels are the same** `hClass` — so `Arch::classify` cannot answer which engine a channel is for, and a CE channel only becomes one later, via its engine object and a refinement pass. A version-keyed *table* does not absorb this; only a decoder that declines to decode does.

The worst instance is §11-O7, which had been marked RESOLVED with the answer that is backwards for our bench. It said `GSP_RUN_CPU_SEQUENCER` "is not implemented, do not emit it" — true at 610, where the executor was deleted; at 580 it is fully implemented, dispatched, and first on the bootup-window allowlist, so an emitted sequencer buffer would execute register writes and falcon resets inside the guest kernel rather than being harmlessly ignored. The RESOLVED status has been withdrawn.

The class, which is the part worth keeping. A *versioned* specification was cited as if it were *the* specification. The fix is mechanical and now normative: every citation carries its tag (`ogkm-580: / ogkm-610:`), an untagged claim is unverified by definition, and line numbers *and paths* never cross tags — the whole `msgq` library moved directories between the two.

★ That fix is being applied to new code and the backlog is getting worse, which is the honest and unflattering measurement. At the previous pin the crates carried **121** bare, untagged `ogkm:` citations and **zero** tagged ones. At this revision there are **56 tagged** citations — the rule is real and new work follows it — and the bare count has risen to **159**, because four crates' worth of new code was written faster than the backlog was converted. `kayfabe-gsp`'s own share fell slightly, 96 to 92. **The CI grep that would stop the class recurring is specified and still not written**, and it is now the only thing standing between a normative rule and a growing pile of counter-examples. The task is filed (correction-brief item B11) and was explicitly not reached.

The self-critical note in the correction log is the one to read: one claim in the same document was marked `[inferred]` and turned out to be right, while O7 was marked RESOLVED and turned out to be backwards — and the difference was not luck. *The inferred claim knew it was second-hand and the resolved one did not.*

What this does and does not change about the critical path

For three revisions this subsection said the gap had moved up one layer and had not closed. It has closed, and the answer was not what any of those revisions predicted. The L2 adapter exists, `GuestRam` has a production implementor, `GraphPolicy` is constructed outside `tests/`, and `DriverAbiTable::gsp_element` has a caller: the shipped register plane selects a wire layout to answer a real guest through. The version-dispatch machinery finally has something production-side asking it a question.

And the critical path turned out never to have been any of the layers this paper kept nominating. It was not the crate, then the bridge, then the L2 adapter. **It is the execution plane** (§8.2) — a plane no amount of protocol modelling reaches, because the guest driver's own acceptance test for it is a functional copy with a read-back comparison. *Read §8.4 with that in hand*: even a complete bridge fed by a complete adapter produces a correct object model and an address table that only one of its two sources fully populates.

★ Worth noticing about this document rather than about the code. Three consecutive revisions each identified "the critical path", each was invalidated within days by the work it named, and *each was invalidated in the same direction* — the named obstacle got built and was not the obstacle. That is a pattern in the paper's own reasoning, not in the project's execution: **a layer that is missing is easy to see, and a plane that cannot be faked is not a layer.** It is recorded here because the fourth answer above should be read with exactly that much suspicion.

8.8 What contact with hardware has and has not bought

Stated once more, in its corrected form, because it is the thing most likely to be half-remembered between here and the end of the paper.

What is no longer true. There is no binary — *false*: there are three, plus a `staticlib` inside a real QEMU.

`RmBackend` and `IsolateFactory` have only mocks — *false*. No isolate process has ever been spawned — *false*: one is spawned in response to the guest's own action, sandboxed and capability-less, and the timestamps are on disk. No guest driver has ever posted to the faked GSP — *false*: a stock 580.159.04 module does, and 92–95 commands decode per boot. `Arch` has zero production implementations — *false*: three.

What is still true, and it is the larger part.

- **The boot does not complete**, on any box, at any revision. `RmInitAdapter` fails; no device node; `nvidia-smi` finds nothing.
- **No compute exists in Mode 2.** No `cuInit`, no `cuCtxCreate`, no kernel launch, no matmul. The one CUDA binary in the tree is a host-side blast-radius probe that never goes through kayfabe at all.
- **The control plane and the execution plane have not met.** Every boot: `doorbells: 0 arrived`.
- **The multi-process property — the founding requirement of the whole rewrite — is [unverified] on hardware.** Every bench arm has exactly one `Proc`, because the guest never reaches a second guest process. #14, the bug that motivated the rewrite, cannot yet be shown fixed on a GPU.
- **The completion plane has been driven by nothing.** No interrupt has been raised for a guest; the C oracle cannot see this plane at all (§3.1, limit (i)); and the one measurement that exists says event delivery is gated off after `INIT_DONE`.
- **The trace vocabulary still does not emit.** A grep for the emit call over `crates/*/src` hits only the crate that defines it.

What this means for the claims in this document. Everything about *structure* — the DAG, the purity, the type-level isolation, the lock discipline — is verifiable today and this paper verified it. Everything about *behaviour against NVIDIA* was inherited from the C artifact and re-expressed, and **the re-expression has now been checked at exactly two points**: a driver binds, and a copy engine copies. Between those two points sits the entire reason the project exists. The gap is smaller than at any previous revision and it is still the gap the C artifact spent ten weeks and 753 commits closing, once, in C.

★ **And the sharpest single lesson of getting this far is about testing, not about NVIDIA.** An R1 invariant was violated on the production path for an unknown number of commits while **1 975 tests were green** (§8.12). The assert existed, was correct, and was old. It was simply unreachable from any path CI can execute, because the suite spawns isolates through a mock that cannot block. *Where the mock and the real implementation differ in whether they can violate a rule, the assert belongs at the seam they share* — and until it does, the suite's green is a statement about the mock.

8.9 The quadratic control plane — an open, measured DoS

The invariant catalogue's I4 says hostile input is *contained*: no wedge, no OOM, no bystander corruption, every hostile event earning only its own refusal. All of that is property-proven. It says nothing about *cost*, and the project measured the cost and recorded the result against itself:

The control plane is $O(\text{live objects})$ per event, so N events cost $O(N^2)$. Measured on a debug build: **1 000 events 0.85 s; 8 000 events 54.8 s**. The capacity cap is 2^{18} live handles, so a guest can drive that curve deliberately — and **PyTorch startup reaches it benignly**. Deleting the per-event graph clone saves ~24% and leaves the curve quadratic, so the clone is neither the cause nor the fix.

The stated statement in `ARCHITECTURE.md` — "no $O(n^2)$ path on hostile floods" — is marked **false** in the same document. Two candidate fixes are named and neither is small: incremental derivation, which is a redesign of the "derived state is a pure function of the graph, never accreted" decision that the entire determinism differential rests on; or an undo journal in the most safety-critical file in the repository. A sibling of the same species *was* fixed (a condemned-list carry-forward, $O(n^2 \log n)$, 55 s → 3.8 s, by union-find), which is evidence the class is tractable and also evidence that it recurs.

8.10 Reclamation is the least finished plane

Figure 5 marks two triggers red. Restating the sharper form:

"Retire eager, reap deferred" is currently half a law. The deferral has no bound and nothing arms it. In a run that never quiesces, reap-deferred and reap-never are indistinguishable.

The one genuine cross-process reference has a disposition that no ledger will like. On a kernel/UVM dup of a user address space, refcount zero and per-object `Free` are not the same event: the last reference retires and reaps the owner *without issuing a single* `Free` *verb*, and the disposition of record is the isolate process's death.

GPA is conserved independently and asserted. But any future reclamation ledger, quota or accounting hook will be wrong on that path first — which the design says, in advance, in its own words.

And the most leak-prone moment has no vocabulary at all until L1-M2 lands. A worker that dies *mid-chain* cannot run its own unwind, so everything allocated before the failure is in no `Orphans` and in no core state. It is unrecoverable and no current test can see it. The design's answer — escalate to killing the isolate, because killing the process is how you reclaim state you cannot enumerate — is correct and is not yet built.

8.11 arm64 is measured for the core and unmeasured for the shell

The full suite *ran* on a GB10 Grace-Blackwell aarch64 host: **372 passed, 0 failed**, identical to x86_64. That is a real upgrade over a cross-compile check, and it covers the concurrency suites, whose memory-ordering behaviour is exactly what a `cargo check` cannot see.

What it does not establish, stated by the project so nobody over-reads it: `getconf PAGESIZE` was 4096 on that host, so the 16/64 KiB page-size rule — the one real pressure point — is still unexercised. The box was a container with no `/dev/kvm` and no `/dev/userfaultfd`, so **no L1/L2 OS-shell behaviour was exercised on ARM at all**. And there are zero VM-capable arm64 machines on the bench provider, so settling the remaining question by experiment requires hardware from somewhere else.

The honest summary is the project's own: *the pure core is arm64-clean by measurement; the OS shell on arm64 is entirely unmeasured*.

8.12 ★★★ R1 was broken on the one path the suite could not see

Invariant **R1** is the one the whole concurrency design is built on: **no blocking call under any lock, ever**. Blocking calls take *zero* ranked locks. It is asserted, not documented — a thread-local lock-depth counter checked at every blocking-verb entry.

It was violated on the production path, and only a real GPU could say so. With the real isolate plane enabled, QEMU **aborted** on the guest's first register write that reached an object allocation: *"R1 no-blocking-under-lock violation: spawning a sandboxed child process while holding rank(s) [0]"*, in a non-unwinding panic, with `kayfabe_shim_regs_write` ← `nvkvm_trap_write` below it — an ordinary guest register write.

The defect in one sentence. Materialising an isolate is a `clone` into six namespaces, an `execveat` of a sealed memfd, a blocking handshake and a chain of host RM ioctls — and it had been placed on the `Ok` arm of the core's apply, which runs under the device write lock. **R1 admits no exception for that, and the design already said so.**

> **★★★ Why the suite was blind, which is the more important half.** The assert that fired is real, correct and old — it lives at the syscall, in the raw-syscall crate. **It is reachable only on the host plane.** The whole suite spawns through `MockIsolateFactory`, whose spawn blocks on nothing and asserts nothing, so on every path CI can execute the violation was invisible: **1 975 green tests over a live R1 breach.**
>
> *An invariant asserted only in the production adapter is an invariant the test suite does not hold.* The repair was not only to move the spawn — it was to move *the assert*, to `IsolateBox::new`, which is the one way an isolate enters core state whichever plane made it. **That turns the entire mock-driven suite into a live guard for it.**
>
> ⊘ **And when it broke is [unverified].** Earlier increments' live real-isolate measurements no longer reproduce, no bisection was run, and no claim is made about which commit did it.

The fix is a phase-shape change, not an edit, and it is the same latch-and-discharge two-step the codebase already used twice elsewhere: *decide* which isolates are needed under rank 0, *spawn* with zero locks held, *install* under rank 0 with R5 re-validation. A new deferred state, `IsolatePending`, resolves in the device loop. ★ **Six doors needed the discrimination, not two**, and *four of the six were found by the test and by nothing else*.

The acceptance, and it is a properly-controlled one. Three boots on an RTX 3060: two with the real plane, both exit 0, **zero** R1 violations, one isolate child each at `t+42 s` and `t+43 s` against a device open at `t+37 s` — so the spawn still *follows* the guest's action, re-measured rather than assumed. ★ And the pre-fix control was taken **on a second physical machine the fix's author never touched**, at a master revision, where the abort reproduces — so the pair is a genuine control rather than a comparison across two variables at once.

⊘ **And the honest headline is that the wall did not move.** The guest dmesg is byte-identical across all three arms — 26 lines, 22 `NVRM`, 3 adapter lines. *Why unchanged is the expected answer and not a disappointment*: the isolate plane serves *verbs*, and the boot dies long before a channel is scheduled, so there is no guest submission for it to serve. What the run buys is that **the plane can be switched on at all**, which is the prerequisite for everything after it.

★ **Five bite-checks, four bit, and the fifth found the test.** A `#[should_panic(expected = "R1 no-blocking-under-lock")]` was *vacuous*: the assert under test shares its panic message with the isolate's own `Drop`, so the expectation was being satisfied by the wrong door. The general form, worth carrying: **a `#[should_panic]` whose expected string is produced by more than one site in the code under test is not a test of either of them.**

8.13 Multi-tenancy: two shapes contained, one that bites, and no tenant axis at all

Multi-tenancy is the thesis (§3), so it is the claim most worth attacking. Three denial shapes were measured on a real GA106, and they did not come back the same way.

Shape	Verdict	What was measured
Infinite compute kernel	**contained**	229 376 threads against a ∼43 008-thread resident capacity, so the device is genuinely oversubscribed. A second process: **12/12** runs over 60 s, `bad=0` every time, **zero Xid**, no reset. It costs its neighbour roughly 2× throughput — *a fairness problem, which is the honest name for it*. ⊘ The *mechanism* is [inferred]; the run observed only that the victim survived.
Guest-reachable MMU fault	**contained**	A genuine `Xid 31 FAULT_PDE`. The attacker's context dies sticky; ★ a victim **holding a live context across the fault** completed **2 682 576 iterations, 2 682 576 correct, 0 wrong**. **Stronger than the row above, because the escalation path actually ran** rather than merely not being needed.
VRAM exhaustion	★ **NOT contained**	An *unprivileged* hog took **11 776 of 12 288 MiB (95.8%)**. A second tenant **cannot even create a context** — out-of-memory in **0.29 s**.

★ **Why only the third bites, and it is a one-line argument.** The first two rest on *the GPU's scheduler*. The third rests on nothing — it is ordinary allocation. **Denial is total**: not "the victim's allocation fails" but "the victim cannot get onto the device at all", which is strictly worse, because a tenant that cannot create a context cannot even begin to degrade gracefully. ★ **But it is honest denial** — immediate, named, and fully reversible, with no Xid and no reset. *That is a resource problem, not a recovery problem*, and the mitigation is ordinary and buildable: a per-tenant VRAM quota, enforced where we already broker allocation. It is the one multi-tenant exposure so far that is squarely inside our reach.

★★ **And the instrument was wrong first, in the way this paper keeps recording.** The first version of the wedge probe span **1 block of 32 threads on a 28-SM GPU**. The victim survived *because 27 SMs were free*. It reported total containment and **could not have shown otherwise** — and the one number that looked like a saturation check, GPU utilisation, reads 100% in both cases, so it could not discriminate either.

★★★ **The confidentiality answer is NO, and the reason is structural rather than a missing control.** Asked whether the security model is sufficient for aggressive multi-tenancy, the assessment answers: **no — because there is no tenant axis in this system at all.** Every isolation property this tree proves is *intra-VM, between guest processes*. There is no `VmId`, `TenantId` or `GuestId` anywhere in `crates/`. Cross-tenant separation is the incidental consequence of two VMs being two host processes.

And at the one place two tenants genuinely meet — the host NVIDIA driver — the driver's own cross-client check is defeated by construction. RM compares client tokens with an **OR**: a matching euid alone passes. Every isolate presents the same euid, so two tenants' isolates pass RM's cross-client check against each other. ★ And the handle is *not* guessable-in-theory-only: it is a fixed base OR'd with a monotonically increasing index, so concurrent isolates receive adjacent values. **There is nothing to guess.**

★ **Two narrowings that are genuinely reassuring, and are not the same as a fix.** RM's default cross-client *dup* policy is PID-scoped, not euid-scoped, so the dup path across tenants *is* refused even at matching euid — the exposure is the direct-naming path only. And the expected break — residual tenant data on GPU memory reuse — **does not exist on the paths we drive**: the host driver scrubs VRAM on free and makes the frame un-allocatable until the scrub retires.

⊘ **Every claim in this box is [inferred] from driver source. No GPU was switched on for it, both source documents say so in their own banners, and the single experiment that would settle it — two VMs on one box — has never been attempted.** The highest-value single change is named and is one line of deployment policy: **run each isolate at a distinct non-zero uid**, which moves the boundary from our code to the kernel's.

So where does that leave "red-teamed at the logic layer only"? That sentence, which this paper printed at the previous revision, is now **half discharged and half still true**, and the split is worth stating precisely.

Discharged at the host-GPU boundary: a system-level property now exists — *every effect a hostile guest can produce on the host GPU is an effect a local unprivileged process could already produce* — with a fourteen-finding audit of the actual host-facing surface behind it, the three denial shapes above measured on hardware, and a **measured host-filesystem escape found and closed** (the isolate's own `/dev` descriptor opened `../etc/shadow` on a real host). **Still open at the cross-tenant boundary**: no tenant axis, **no** `seccomp` **anywhere in the tree** (absent, and named as absent rather than stubbed), no two-VM run, and the security review of the new FFI surface is the declared exit gate of the L2 work and **has not been done**.

8.14 The QEMU $\geq$ 10.2 floor buys less than the decision that took it assumed

★ **This section is no longer a forecast.** At the previous pin the L2 adapter had not been started and this section priced it. It is now built through stage Q5, compiled into *two* real hypervisor binaries (9.2.0 and 10.2.4, the same overlay unchanged), and booted against repeatedly on a real GA106 — and on 9.2.0 the identical binary path is **refused by name** at the runtime floor, which was watched rather than assumed. **Every cost itemised below was paid, and none of the estimates below turned out to be wrong.** The exit gate, Q7 — the security review of the FFI surface and the written R1/R3/R5 re-audit — is **not done**.

The design requires `memory_region_enable_lockless_io()`, which lands in QEMU 10.2. That buys a genuinely important thing — MMIO dispatch without the big QEMU lock, which is what makes the doorbell path viable — and upstream's helper also sets the reentrancy guard in the same body, so a discipline the design planned to maintain is no longer ours.

The floor decision's second claim was economic: requiring a version rather than patching one deletes *"the whole 'which QEMU are we built into' ambiguity"*. **The L2-Q adapter design** (`b31487a`) **checked that claim against QEMU's source and it does not hold.**

QEMU v10.2.0 has no supported out-of-tree device mechanism, so our device must be compiled inside a QEMU source tree. Two mechanisms, both closed: `module_load_qom()` resolves a QOM type name against `module_info`, a table *generated at QEMU build time*, so a `.so` that is not in that table can never be discovered by type name — and `QEMU_MODULE_DIR` changes only the directory searched, not the table. `module_load_dso()` additionally requires the module to export a per-build `CONFIG_STAMP` symbol; upstream's own failure hint is verbatim *"Only modules from the same build can be loaded."* [v10.2.0 util/module.c, include/qemu/module.h]

State it exactly, without overstating it. The floor *does* delete a patch we would otherwise have had to carry: the cancelled 9.2 backport modified upstream semantics inside `system/physmem.c`, had to be re-derived every release, and a mis-rebase would silently change how *every* device in the machine dispatched. What replaces it is an additive overlay — a directory dropped into a stock 10.2.x checkout plus two hunks (one `meson.build` line, one `Kconfig` stanza) — whose only conflict surface is a build failure, never a behaviour change. That is a real improvement. **What it does not buy is "install your distro's QEMU and run".** The user builds QEMU once. Three of the four costs the floor decision itemised against the backport — a rebase every release, a build script that must reproduce it, and a supply-chain claim we make to every user — still apply.

There is exactly one path to a genuinely stock QEMU, it is named, and it is not taken for v1. `vfio-user` would make our device a separate process speaking a documented socket protocol to an unmodified distro-packaged QEMU. It is present at v10.2.0 (`hw/vfio-user/`, two documentation files, a vendored `libvfio-user`) and `MAINTAINERS` lists it `S: Supported`. It would delete the QEMU tree, the overlay, and the second `unsafe` crate the in-tree route needs. It is rejected for v1 on one source read: nothing in `hw/vfio-user/` calls `memory_region_enable_lockless_io`, so a trapped BAR access becomes a socket round trip *taken with the BQL held* — reintroducing and amplifying the exact serialisation the floor was taken to remove. The design records the small experiment that would change the verdict (measure the round trip both ways, and ask whether marking the client's regions lockless is a patch upstream would accept) rather than closing the question.

The facility inventory that priced the floor also **reversed two of its own design assumptions**, which is worth knowing before trusting the rest of it: the read-only-memslot region-lock fallback, priced in the design at 1.49 µs as "a flags-only flip", is *unreachable through QEMU* — QEMU's flatten pass turns a readonly change into a delete-plus-add. And checkpoint-restart (`cpr-transfer`) is a lifecycle event the design had never considered.

One more cost the adapter design surfaced, did not argue away, and then paid. A QEMU adapter needs `extern "C"` entry points, raw `MemoryRegion *` handles and adoption of a host pointer QEMU owns — all `unsafe`. The workspace's CI gate exists to keep the audit surface at *one* crate readable in one sitting, and its own failure text says adding a second is "a design decision, not a manifest edit". **That decision was taken and the price was paid in full**: `kayfabe-qemu-raw` is the second audited `unsafe` crate at **47** relaxations, the ratchet is now a

named two-element list, and the acceptance criterion the gate protects *is* "read two crates". ★ Its bar started at **0** deliberately, so the first relaxation had to be a reviewed commit rather than a manifest edit — and the crate's first act as an FFI consumer was to prove the ratchet's own counting pattern wrong (§5).

And the packaging verdict is unchanged and now testable. What the floor buys is an additive overlay — a directory dropped into a stock 10.2.x checkout plus two build-file hunks — and that is exactly what shipped: `qemu/hw/misc/nvkvm/`, 2 676 lines of C, built by `scripts/build_qom_shim.sh`. **What it does not buy is still "install your distro's QEMU and run".** The user builds QEMU once. The `vfio-user` route that would delete that cost remains named, priced and not taken for v1.

8.15 The bench — rented, serialised, and now more than one box

All hardware work happens on rented GPU boxes, strictly serially: concurrent or mid-ioctl activity wedges the GPU. Two cargo builds on one box corrupted a mutation run through the shared `~/.cargo`, so the rule is one cargo per box.

★ **The previous revision said "there is no second machine, no second GPU generation, and no continuous hardware testing." Two thirds of that is now false and the correction matters, because $n = 1$ was this paper's standing caveat on every hardware number.** Work since has run on **four machines and three distinct physical GPUs**: a GPU-less four-core development box (where, notably, *every* guest boot in the boot log ran); a rented 38-core box with an **RTX 3090 (GA102)**; and **two different RTX 3060s (GA106)** on separate rented instances. Three cross-machine reproductions are load-bearing rather than decorative:

- A wall reproduced **line for line, 36 of 36 dmesg lines**, between the GPU-less box and a real-GPU box — which is what establishes it is a property of the port and not of a machine.
- The bring-up ladder run on **two different physical GA106s** differs by **one line of thirty-three**, and that line is a handle. A committed transcript is therefore a real cross-machine regression test.
- The R1 abort (§8.12) was reproduced at a master revision on **a second machine the fix's author never touched**, which is what makes its before/after a control rather than a comparison across two variables.

⊘ **What has *not* changed, and the reader should hold onto it.** Every *boot* result is GA106, one driver family (580.159.04 open), one guest OS, and one guest at a time. The GA102 appears only in ladder and suite runs, never in a boot. There is still **no continuous hardware testing** — GitHub CI has no GPU and never will. And the project's own standing caveat is repeated verbatim in every boot section for a reason: **one Mode-2 symptom was 1/3 one day and 9/9 the next on a bit-identical binary**, so a single green boot is weak evidence and the documents treat it that way.

★★ **And the bench has lied about its own provenance, more than once, which is why every claim here carries a revision.** A binary served for weeks was built from a commit nobody had noticed. A later one **could not name its own revision at all**: it contained exactly one 40-hex string, which was a build-id and not a commit in either repository, and its only provenance was a worktree **19 commits behind master**. The instrument that was supposed to discriminate — `strings` for feature markers — scored **0 for every marker in both binaries**, because a release build strips enum names and inlines constants, and that was only discovered by checking it against a known-good symbol. ⇒ `strings` **cannot answer "what revision is this"; embed the SHA** — which the archive now does.

8.16 The recurring failure mode: claims that were true *once*

This is the pattern the project keeps hitting, it has a name in the repository, and it is the single most useful thing for a reviewer to internalise.

> Across three independent passes, roughly **twenty** instances were found of documentation asserting more than the code did — **several stating the literal opposite of their own code**. **Every one of them was true when written.** That is the point: this is not carelessness, it is entropy. [docs/design/testing_doctrine.md §6]

The instances are specific and they are not small:

- A design document named `memory_region_clear_global_locking()` as the mechanism for a decision. **That QEMU function had been deleted five years earlier.** The reference document written to price the floor now carries a `file:line` on every row for exactly this reason — and found two *more* live instances of the same decay while being written.
- A refactor plan cited a userfaultfd proof-of-concept file **that does not exist**. The mechanism was planned four times and never built; its demand-fault path is dead code that calls `stub_exit(139)` and never returns,

so the fault address has been permanently zero since a commit early in the project's life. Nobody noticed because "planned" and "presumably done" read the same in a plan.
- **Two sections of one design document contradicted each other**, found only when the code between them was written.
- Two claims about the C's behaviour, cited from the C's own source comments, turned out to be **false when measured on the bench**. The rule extracted: citing a comment is citing a belief.
- A rustdoc described a default-to-GPU-0 guess *inside the one resolver whose entire discipline is "never guess"*. Another claimed "MISS=FAULT is preserved — never a silent skip" while its own body correctly skipped on both arms; the body was right and the sentence was wrong about the most-cited exception in the codebase.

The project's response is the right one: prefer a mechanism to a sentence. A rule stated only in prose has no reader. `HostHandle` carrying its `IsolateId` is the same rule with a compiler behind it. The house style for corrections is *strike visibly* — keep the original, struck, with the correction and its evidence — so a reader who remembers the old claim learns it was wrong rather than doubting their memory.

And the pattern is live right now, in this document's subject, and in this document. The first revision found fourteen instances (§10 rows 1–14). The second found six more, two of them in the first revision's own text. The third found six more again, *three* in this paper including its headline. **This revision found eight more, and *five* of them are in this paper** — including, again, the sentence it led with, which has now been the falsified claim in **three consecutive revisions**.

> ★★ **The direction of the drift inverted at this revision, and that is a new thing to say about it.** Every previous instance was a document claiming *more* than the code did. Five of this revision's eight are the opposite: the paper claimed *less* than the code did — no binary, no isolate, no `Arch` implementation, no walk function, one adapter. **An understatement decays exactly as fast as an overstatement and is harder to notice**, because nobody audits a document for being too modest, and a reader who acts on it makes a *different* wrong decision rather than an obviously wrong one. ⊘ It is the same defect: a description that used to be true of a thing.

Every claim that *does* have a gate is mechanically true today — the boundary greps, the generation-name grep, the two-crate unsafe ratchet, the seven reached-count floors, the "exactly one GPA call site" assertion, the E0639 compile-fail row that makes the ring gate structural, the bridge-exclusivity grep *which fired on its first run*, the ogkm-version-tag gate, and the claim ledger. Four revisions of this paper have now found roughly thirty-four ungated claims to be stale and **zero** gated ones. That is either a devastating footnote or the strongest possible evidence for the project's own thesis, and it is the latter.

★ **With one qualification this revision earned the hard way, and it cuts against the sentence above.** A gated claim is mechanically true *of what the gate quantifies over*, and three separate instruments were found this cycle to be quantifying over less than anyone thought: the mutation gate scored 100% over a universe that a full disk had eaten (§7.7); the unsafe ratchet's own regex could not match the dominant form in the crate it was newly guarding, and **counted 23 of 31 while calling itself a complete audit** (§5); and a citation gate was satisfied by a source that said nothing (§3.1). *"Zero gated claims were stale" is a true statement about a set whose size is itself a measurement* — and all three of those measurements were wrong upward.

★ **The sharpest form of it at this revision is not in the prose at all — it is in the instruments** (§8.6). A configuration file that was never read, a baseline that omitted the suite, a timeout scored as a kill and a hang that printed nothing are all the same species as a stale sentence: a description that used to be true of a thing. The difference is that those four described *the evidence*, which makes them the more expensive kind.

The strongest single instance is in §8.7 and it is worth restating here as a pattern rather than a fact about NVIDIA. A specification was cited as if it were the specification, when it was one version of a specification; 3 550 lines were built against it; and one question had been closed with an answer that is exactly backwards for the machine the code will first run on. The claim that survived that audit was the one marked `[inferred]` — because it knew it was second-hand. The claim that did not survive was marked **RESOLVED**.

8.17 Risks this document adds

- ★★★ **The execution plane cannot be faked, and it is the whole remaining problem.** §8.2. The guest driver demands a functional copy-engine round trip with a read-back comparison, every escape hatch is closed by citation, and one of them is closed *structurally* — the simulation flags are declared and never assigned, so the lie cannot be told. **Everything this project has built for the control plane does not reach this.** E0–E7 have carried one guest-shaped request across it on real silicon; no guest has.
- ★★ **The two working halves have not been joined, and nothing yet dates the join.** A guest drives the

control plane; a harness drives the execution plane; `doorbells: 0 arrived`. Anyone reading either result as "nearly compute" is reading it wrong.

- **The mocks are still the only implementation of `Device`, and a mock is where an invariant hid for an unknown number of commits.** §8.12 is the measured demonstration that this is not a theoretical concern: 1 975 green tests over a live R1 breach, because the double could not violate the rule. **If a belief about RM semantics is wrong, the suite will confirm it consistently** — and if a rule is only assertable in production, the suite does not hold it at all.
- **The project moves faster than its own documentation, and it has not slowed.** 272 commits since the previous pin. `crate_maturity_map.md` — a document whose entire job is to say what is built — is now wholly stale, and this paper stopped citing it rather than laundering numbers through it. **No gate covers prose**, which is why §10 keeps growing.
- ★ **Multi-tenancy has no tenant axis, and that is the thesis.** §8.13. Two denial shapes are contained and one is not; the confidentiality answer is *no*; the host driver's own cross-client check is defeated by a shared euid; and **the single experiment that would settle any of it — two VMs on one box — has never been attempted**. The highest-value fix is one line of deployment policy, which is encouraging and is not the same as having done it.
- ★ **The multi-process property is unmeasured on hardware**, and it is the founding requirement of the entire rewrite. Every bench arm has exactly one `Proc`. Bug #14 — the reason this codebase exists — cannot yet be shown fixed on a GPU.
- ★ **The bridge is built, it now has a production consumer, and it is still not on the path to compute.** §8.4: `MAP_MEMORY_DMA` is not on the wire on any GSP-client part. One of the two real populate sources is built; the other is **partial**, and says so through a test that goes red the day it stops being partial.
- ★★ **Every mutation number this project has published has been wrong upward, five times, by five different mechanisms** (§7.7). The most recent scored **exactly 100%** while a full disk deleted 78% of its universe and the harness reported `ok`. A bar of 91 is now in force; **no clean campaign has completed under it**, so there is still no defensible score to quote.
- **The C oracle is positively wrong about a whole class of rows, and a third class is unmeasured** (§3.1). 16 truncated rows — more than the 11 empty ones — have never been checked, and a truncated row passes the test built to catch the empty ones.
- ★ **The security review of the new FFI surface is the declared exit gate of the L2 work and has not been done**, while the surface itself is now 47 audited `unsafe` relaxations in a crate a hypervisor calls directly. There is no `seccomp` anywhere in the tree.
- **One open item bears directly on a stated product requirement and has no proposed resolution.** §11-O7a (§8.7) may mean that "GPU restart works with no bolt-on" is not satisfiable at the bench's driver version without version-conditional behaviour that the project's own rules forbid in a logic crate. It cannot be closed from open source. Everything else in this list is a known quantity with a known fix; this one is not yet a known quantity.

9 How to read the code

9.1 Entry points

If you want to understand...	Read
the whole system, fastest	`README.md`, then `ARCHITECTURE.md` — but read §10 of this paper first, because both are stale in specific ways
what a "process" is and why	`crates/kayfabe-core/src/project.rs` — the pure derivation
the source of truth	`crates/kayfabe-core/src/rmgraph.rs` — the refcounted graph and the parked-fact machinery
the transactional apply	`crates/kayfabe-core/src/gpu.rs`, `Gpu::apply` → `Spine::apply`
the forwarding entry points	`crates/kayfabe-fwd/src/lib.rs` — start at `route_doorbell`, then `plan_doorbell`; the ring gate itself now lives one crate over, at `kayfabe-isolate` `VerbPlan::gated_doorbell`, with its compile-fail pin in `crates/kayfabe-isolate/tests/ui/`
the faked GSP	`crates/kayfabe-gsp/src/lib.rs` (the module map and the five seams), then `boot.rs` — `BootPhase`×`QueueState` is the whole boot protocol; the test-side guest is `tests/src/gspworld.rs`
why the project is not finished	`docs/design/execution_plane_increments.md` §0 and §10, then `docs/design/boot_measured_2026_08_01.md` §44 — the first says what the execution plane costs, the second is 1 754 lines of dated boot log ending at a wall that is not a control
what the hypervisor actually runs	`crates/kayfabe-qemu-raw/src/shim.rs` and `qemu/hw/misc/nvkvm/nvkvm.c` — the Rust `staticlib` and the C QOM shim that links it
the host side	`crates/kayfabe-isolate-host/src/rm.rs` — the real RM verbs, and `CeEvidence::copied()`, the predicate the whole execution plane is accepted against
the GSP → core bridge	`docs/design/gsp_core_bridge.md` §2.7 *first* (it is the finding that changed the roadmap), then `crates/kayfabe-rmrpc/src/lib.rs` `translate` and `BridgeRefusal` — the refusal enum is the honest map of what is not modelled
the concurrency design	`crates/kayfabe-rt/src/lock.rs` (ranks) then `device.rs` `verb_op` — the whole plan/execute/commit shape is one function
the `unsafe` surface	`ls crates/kayfabe-linux-raw/src/*_unsafe.rs` — that is the complete list, by construction
what a test is supposed to look like	`tests/tests/l1_mean.rs` (the arbiter) and `tests/tests/miss_taxonomy.rs` (the sharpest single property)

9.2 Conventions

- `*_unsafe.rs` — any file using the `unsafe` keyword is named this way, and the gate greps whole words so writing *about* the rule (`unsafe_code`, `_unsafe.rs`) never trips it.
- `plan_* / commit_*` — the two halves of the verb seam. `plan_` runs under locks and issues nothing; `commit_` runs after re-acquisition and re-validates.
- `route_* / exec_*` — the route/act split (structural rule R4): resolve identity against device-wide state, then act on one process's state.
- Test names carry their provenance: `t12_`, `t13_`, `t14_` are C-bug regressions; `cb*` are the regression matrix rows; `b1_`–`b6_` are security boundary cases; `i1`–`i4` are the isolation properties; the marker *Mutation-gate kill* in a doc comment names the mutant a test exists to kill.

9.3 Which design documents to read, in order

1. `docs/design/testing_doctrine.md` (321 lines) — read this *first*, even before the architecture. It is the shortest path to understanding what the project believes evidence is.
2. `docs/design/core_state_and_consolidation.md` — the reviewed per-crate state and the L1 hand-off contract.
3. `docs/design/core_security_threat_model.md` — the attacker model and the I1–I4 properties.
4. `docs/design/l1_concurrency.md` §12 and `docs/design/l1_os_shell.md` §14 — the contact logs (the two documents run to 10 265 lines between them). **Read these rather than the summaries**; they are where the design was found to be wrong, which is the part a summary cannot carry.
5. `docs/design/execution_plane_increments.md` — **read §0 and §10 before forming any view of how close this project is to compute.** §0 records the ordering judgement the boot refuted; §10 is the hardware acceptance and

the wall that replaced it.

6. `docs/design/boot_measured_2026_08_01.md` — 1 754 lines of dated, revision-stamped boot log. It is the least summarisable and most load-bearing document in the repository, and it contains at least three sections marked **REFUTED** that were kept rather than deleted, each with the reason the inference was wrong.
7. `docs/design/guest_blast_radius.md` and `docs/design/multi_tenant_isolation_assessment.md` — the security position on the thesis. Read the banners first: neither switched on a GPU for its reasoning.
8. `docs/reference/` — measured ground truth, kept out of the design documents so a wrong fact is corrected in one place. Note the version caveat in `rm_semantics_measured.md` §0 before quoting any number from it.
9. ⊘ **Do not read** `docs/design/crate_maturity_map.md` — §10 row 29.

10 Claims corrected while writing this paper

Each of these is a claim that this paper checked against the tree and found stale or wrong. They are listed because the list is more useful than any individual correction: it is a measurement of the drift rate described in §8.16. None is a defect in the code; all are defects in descriptions of the code.

Rows 1–14 are from the first revision, at `0b78102`; rows 15–20 from the second, at `6ce8599`; rows 21–26 from the third, at `b612a91`; **rows 27–34 are new at** `f33e1c8`. **Eleven of the first fourteen were fixed or superseded within four days**, and each row says which — that is the loop working, and it is the single most encouraging measurement in this paper. Row 9 is the instructive exception: it was fixed *in one document*, which then recorded four further documents still citing the retracted number and declared them out of scope.

Five of this revision's eight rows are defects in this paper's own previous revision, and one of them is — for the third consecutive time — the sentence the previous revision was built around. That is the point rather than an embarrassment to be minimised: a whitepaper is a description of code and decays at the same rate as any other description of code. The claims that decay fastest are the ones stated most confidently, because confidence is what stops the next reader re-checking them.

★ **And two of this revision's rows are a new species: numbers supplied to the writer in the brief for this revision, which were checked and found wrong.** They are listed at 31–32 with the same weight as the rest. An instruction is a claim like any other, and the only reason they did not enter the document is that the method (§A) re-derives rather than restates.

	Claim, and where	What the tree says
1	*"*`handle_doorbell` *is the ONLY caller of* `RmBackend::ring_doorbell`*"* — `ARCHITECTURE.md:112`, `core_completeness_gate.md:119`, `core_state_and_consolidation.md:174`, `fwd/src/lib.rs:15`	**False as stated.** `ring_doorbell` has exactly one call site and it is inside `Worker::execute`, two hops away. `plan_doorbell` had *two* callers: `handle_doorbell` and `SharedDevice::doorbell` — and the L1 shell path, the one a real guest MMIO write takes, goes through the second and **never enters** `handle_doorbell`. The safety property survived; the cardinality claim did not. **Superseded at** `634b253`: the gate moved into `VerbPlan::gated_doorbell`, the sole constructor of a `#[non_exhaustive]` variant, so the property no longer rests on a cardinality claim at all — it rests on E0639. *A correction that the code then made unnecessary is the best outcome on this list.*
2	`kayfabe-abi` is a *"stub (shape only)"* — `README.md`, `ARCHITECTURE.md`, and its own `Cargo.toml` description	**False**, and **FIXED SINCE** (`e6ba19e`). 7 455 lines of source, a working offline generator, 11 generated structs, a 13-method version-dispatch decode surface, 2 272 lines of oracle tests. All three sites now carry struck-through corrections. *Listed here because the correction is the evidence the house style works.*
3	*"the implementor is the L1 shell* (`kayfabe_rt::SharedDevice`)*"* — `ARCHITECTURE.md:38--43`, `kayfabe-vmm/src/lib.rs:816`, `core/src/gpu.rs:1012`	**No** `impl Device for SharedDevice` **exists.** The only two implementors are test types that wrap an `Arc<SharedDevice>` and forward. **Still true at** `6ce8599`, but **the descriptions are FIXED**: both `ARCHITECTURE.md` and `kayfabe-vmm/src/lib.rs` now state the absence in bold rather than implying the shell already does it.
4	*"283 tests"* — `README.md`, `ARCHITECTURE.md`	**515** measured on 2026-07-27; **547** at this revision. **FIXED SINCE** (`e6ba19e`) — and fixed in the right way: both files now say "in the **500s** as of 2026-07-27" rather than carrying a fresh exact number that would rot again in a week. *Deleting a number is a legitimate repair.*

	Claim, and where	What the tree says
5	*"four standing CI gates"* — `README.md`	**Six jobs; twelve steps in the `stable` job alone.** The list of four omitted the VMM-vocabulary gate, the GPA-accessor gate, three unsafe-containment asserts, the KVM reached-count floor, the fuzz-compile job, the slow job and the mutation job — and, since it was written, the generation-name gate. **FIXED SINCE**: the phrase no longer appears in `README.md`.
6	*"TSan (28 threaded tests, 0 races)"* — `README.md`	28 was the first campaign. The four TSan targets contain **68** `#[test]` functions at this revision (65 at the previous one). The "0 races" half is not disputed here — it was not re-run. **FIXED SINCE**: `README.md` now names 28 as "the first campaign" and says it has not been re-run.
7	*"the two measured-slow tests"* — `README.md`	Four to six, depending on how you count; the `g10_*` capped pair joined later. **FIXED SINCE**: the README now says membership "has grown since it was measured" and points at the `skip_slow!` call sites as the authority instead of quoting a count.
8	*"the conformance suite (23 files)"* — `ARCHITECTURE.md`	**29** files in `tests/tests/` at this revision, plus **6** under `crates/*/tests/` (28 + 5 at the previous one). **FIXED SINCE**: the count was removed rather than updated.
9	`core_mutation_gate.md`'s 91% threshold and its *Reproduce* block	Both described the pre-2026-07-27 four-path scope. **FIXED SINCE** (`e6ba19e`): the document now opens with a banner saying so. *But the correction propagated only one hop* — the same document records that the 92.44% and the 91% floor are "still cited as settled" by `README.md`, `ARCHITECTURE.md`, `l1_architecture_summary.md` and `core_state_and_consolidation.md`, and reports them as out of its own edit scope. A correction that names its own unpropagated instances is the honest form; it is still four live instances.
10	`kayfabe-mmu::walker`'s own doc: *"Also here (skeleton this milestone): the GMMU walk algorithm"* — `mmu/src/lib.rs:30`	There is no algorithm: one trait, one enum, 41 lines, zero constructors, zero implementors — **unchanged at** `6ce8599`. **The doc was FIXED** first: the module comment carried a struck-through correction naming what it actually contained. *The code did not move; the claim did.* ★ **And then the code moved too** — see row 30: the module is now 863 lines with six public functions. *This is the only row on this list that was resolved from both ends.*
11	`kayfabe-gsp`'s doc: RPC decode *"against `kayfabe-abi`'s generated message layouts"*	**Was false; is now true.** At the previous pin `kayfabe-gsp` did not depend on `kayfabe-abi` and nothing did. At `6ce8599` the manifest lists it and five of the eight modules use it. *A doc claim that was aspirational and became correct is still a drift instance — nothing gated it either way.*
12	*"`Isolate/RmBackend/Vmm` trait-only (mock-implemented)"* — `ARCHITECTURE.md:31,56`	True for `Isolate/RmBackend`. **False for** `Vmm`: `crates/kayfabe-vmm-kvm` is a real KVM adapter with a real memory plane. **FIXED SINCE** — `ARCHITECTURE.md:31` now carries the struck-through original.
13	*"~14 months of quirk capture"* in the C's architecture and ABI documents	Git: 2026-05-24 to 2026-08-01. **Ten weeks, 753 commits** (it was "two months, 739" when this row was first written; the C is still maintained as the oracle, so the number moves).
14	The framing "7B LLM at 63 tok/s" as a headline C result	63.4 is a Mode-1 A/B endpoint. The audited parity figure is **0.97×** (**66.0 guest vs 67.8 host**). The underlying milestone note still records the pre-fix **20 tok/s** and was never updated. All three numbers are real; they are different builds.
15	`kayfabe-gsp` is a **stub** — `README.md:161`, `ARCHITECTURE.md:68`, and its own `Cargo.toml` description, which still opens *"SKELETON — the faked GSP…"*	**False.** 3 550 lines, 8 modules, 24 dedicated tests, an independent 1 133-line guest model, and a composed run inside the arbiter suite. The two markdown rows at least carry an `[unverified]` hedge telling the reader to read `lib.rs` instead; the `Cargo.toml` description carries none. *This is the same shape as row 2, one crate later, four days after row 2 was written.*
16	`mode2_gsp_port_plan.md` §11-O7 marked **RESOLVED**: *"`GSP_RUN_CPU_SEQUENCER` is not implemented, do not emit it"*	**Backwards for our bench, and the status has been withdrawn** (`7af23cb`). That is 610's answer. At 580 it is fully implemented, dispatched, and first on the bootup-window allowlist. The whole plan's citations were taken from 610.43.02 while the bench runs 580.159.04; eight claims changed when the matching tree was vendored. See §8.7.

	Claim, and where	What the tree says
17	The plan's §1.3: *"the guest declares its own geometry"*, presented as the highest-leverage design choice	**610 only.** At 580 `MESSAGE_QUEUE_INIT_ARGUMENTS` has four fields, not nine, and the queue geometry is compile-time. On the bench nothing is negotiated, so the argument the design leaned on does not apply to the machine it will first run on.
18	**121** bare `ogkm:` citations in the crates' doc comments, 96 of them in `kayfabe-gsp`	**Every one is a 610 path with a 610 line number**, and several name files that do not exist at 580 — the whole `msgq` library moved directories between the tags. The version-tagging rule that fixes the class is normative in the plan and **not applied to the code**; the CI grep that would prevent the recurrence is specified and not written.
19	**This paper's own previous revision:** *"*`kayfabe-abi` *and* `kayfabe-gsp` *are disconnected islands"*	**Half false at** `6ce8599`**.** `abi` has a consumer now (`gsp`); `gsp` still has none outside `tests/`. Re-checked against the manifests, not inherited from the caption.
20	**This paper's own previous revision:** Figure 2, *"all 65* `[dependencies]` *edges are shown"*	**The manifests had 66.** The edge `tests` → `rt` was omitted, in a figure whose stated method is "derived from the manifests". A drawn matrix is exactly as ungated as a sentence.
21	**★★ This paper's own previous revision, its headline:** *"*`RpcCommand` *has zero references anywhere in the tree outside the crate that defines it … the critical path is the bridge, not the crate"*	**False at** `b612a91`**.** `kayfabe-rmrpc` (`76a0077–3a1704f`) names it in three files and is a production crate depending on both `gsp` and `core`; a CI grep now enforces that it is the *only* crate permitted to. *The claim was true when written and was invalidated by the work it identified as necessary* — which is the best possible outcome for a criticism, and still a stale sentence four days later. The successor claim, stated so it can be checked the same way: **nothing but** `tests/Cargo.toml` **names** `kayfabe-rmrpc`.
22	**This paper's own previous revision:** *"Checked-in proptest failures:* ***0*** *—* `find` *for* `proptest-regressions`*: nothing exists"*	**Wrong when printed.** Four `*.proptest-regressions` files existed at `6ce8599` holding **18** seeds; there are 20 now. The row's stated method was a `find` that would have returned them. *This is the only correction on this list that is a measurement error rather than decay*, and it is the one that most deserves to be here: the method section claimed a command was run, and the number it reports is not the number that command produces.
23	**This paper's own previous revision:** the GSP-S1 row, *"Nothing. It is a design-doc invariant with no code behind it …* `grep` *finds the variant nowhere"*, and the risk-list item calling it "the highest-stakes sentence it has"	**Fixed since** (`250b78c`). `GspFault::ElementCountOutOfRange` exists at the write site, unconditionally, plus a receive-side range check and a derivation cross-check. Noted here rather than silently updated, because this paper named the gap and the gap closed — that is the loop working on the paper's own output.
24	`gsp_core_bridge.md` §4.3: *"known-and-inert versus unknown — there is no third state"*; §5.2's B2/B4 non-vacuity arms; §6's Memory-alloc row; §7.1's assumption that `SET_PAGE_DIRECTORY` carries the PDB	**All corrected by the build, in the commit that hit them.** There are at least four dispositions, not two. Three non-vacuity arms were unbuildable at the stage that specified them and one was *wrong* (refusing a second PDB requires state a stateless bridge cannot have, and `RmGraph` had already decided the other way with an argument). The Memory row is unbuildable twice over — the address fields are `[OUT]` in the guest→GSP direction, and its only consumer is driven by an event with no producer. §7.1 is settled against the design: see §8.4.
25	`mutants.toml`'s own header, and every "excluding scaffolding" figure this project has published, including one in this paper's previous revision	**★★ The file was never read** (`b64a3f5`). It lived at the workspace root; cargo-mutants reads `.cargo/mutants.toml`. Every exclusion was inert and every scaffolding-excluded score was a hand subtraction. Proven, not inferred: `--list` targets 3 760 → 2 667, scaffolding mutants 361 → 0. *An inert configuration file is the same failure as a stale sentence, with worse consequences.*
26	*"121 bare* `ogkm:` *citations, and the version-tagging rule is normative"* — §10 row 18, at the previous revision	**The rule is being followed by new code and the backlog grew.** 56 tagged citations now exist where there were none; bare untagged ones went **121 → 159**. The CI grep that would stop it is still specified and not written, and it is now the difference between a rule and a wish. **★ FIXED SINCE**: the grep exists — an *ogkm-version-tag* gate, ratcheted at 161 untagged and exact in both directions. *This paper named the gap twice and the gap closed.*

	Claim, and where	What the tree says
27	★★★ **This paper's own previous revision, its headline and its warnbox:** *"No part of the Rust codebase has ever driven a real GPU … there is no binary in the workspace … no isolate process has ever been spawned … no guest driver has ever posted to any of it"*	**Every clause is now false**, and this is the third consecutive revision whose headline was the falsified claim. A stock 580.159.04 module binds to the emulated device and 92–95 commands decode per boot; three binaries exist plus a `staticlib` in a real QEMU; a sandboxed isolate is spawned at `t+42 s` in response to the guest's action; and a real host copy engine has moved 4 096 bytes for a request this core decoded. ⊘ **And the replacement is deliberately a conjunction, not a sentence** (§8.8): the boot still fails, there is still no `cuInit`, and the two hardware results have not met.
28	**This paper's own previous revision:** *"Defect #57 … trips roughly 1 in 6 runs under race. This is why the QEMU adapter has not been started"* — and two more sites	**False twice over.** #57 was closed at `020790c`, and *the shape of its resolution is the part worth keeping*: the violation was not the lock split or a syscall in a critical section but `Arc` ownership — **a `Drop` is a call site**. And the QEMU adapter is built through stage Q5, compiled into two real hypervisor binaries and booted against. ⚠ **Do not conflate #57 with §8.12**: different crates, different defects, different lessons.
29	`docs/design/crate_maturity_map.md`, cited by this paper as the cross-check for its own maturity column	**Wholly stale, and this paper stopped citing it.** Pinned at `d65eb75` ("496 tests green"); lists 16 crates against 24; calls `kayfabe-gsp` a **34-line skeleton** against 5 056 lines; records #57 open five days after it closed; and names neither QEMU crate. *A document whose entire job is to say what is built is the worst possible one to be stale*, and it is the sharpest instance of §8.16 in the tree.
30	**This paper's own previous revision:** *"the* `walker` *module is 41 lines containing one trait and one enum, with no walk function of any kind"*; and *"still zero production implementations … that remains the standing proof that a real one will need no core edits"* (`kayfabe-arch`)	**Both closed, and both by this paper naming them.** `walker` is 863 lines with six public functions and `FbRead` has two implementors. `Arch` has three production implementations — and the "standing proof" was settled *partly against itself*: Ada needed no seam, Hopper needed one, and the seam it needed was in `kayfabe-arch`. *A standing proof that is never executed is a hypothesis.*
31	★ **The brief for this revision:** *"84 of 106 classified DATA/ACT with citations"*	**The number 84 does not appear in any of the source documents**, and re-derived from `gsp_control_classification.tsv` directly the figures are **79** DATA-or-ACT and **81 of 106 (76.4%)** bucketed with a citation, cross-checked two ways (bucket column and confidence column, HIGH 78 + MEDIUM 3). **25 are UNKNOWN.** The measured numbers are what this paper prints.
32	★ **The brief for this revision:** *"every* `dlen=0` *row checked against a real GA106 is contradicted"*	**Too strong, and the exceptions are the interesting part.** All eleven were checked and **nine** are contradicted. `0x20800a70` has `psize = 0` so nothing could have failed to be captured; `0x20800a4c` genuinely answers zero — **and nothing about the row says so**, it being byte-identical to the nine that are wrong. *Believing the empty rows would have been right once and wrong nine times for the same reason*, which is a strictly better argument than the blanket one. §3.1.
33	**This paper's own previous revision:** *"there is no second machine, no second GPU generation, and no continuous hardware testing"*	**Two thirds false.** Work has run on four machines and three distinct physical GPUs, including *two different* GA106s and a GA102, and three cross-machine reproductions are load-bearing rather than decorative (§8.15). **The third clause is still true.** ⊘ And every *boot* result is still one GPU model, one driver family, one guest.
34	**This paper's own preamble**, and it was introduced by this revision	★ **The glyph ⊘ was being dropped silently from the PDF.** The preamble carries an explicit fallback for ★ and ⚠ with a comment saying a dropped marker is *"a lost signal, not a cosmetic defect"* — and the marker added to the vocabulary since then hit the identical trap, visible only as a `Missing character:` line in a log nobody reads. ⊘ is this project's marker for *"this does NOT establish X"*, **so a dropped one inverts the sentence it qualifies**. Found by reading the build log rather than the output. *The mechanism generalises past this document: a fallback table is a hand-list, and a hand-list is a smaller true statement.*

10.1 What this paper could not verify

- **Any claim requiring a build or a run. No `cargo` command was executed while writing this, at any of the four revisions, and none was run for this one — deliberately, on a disk-constrained box.** Test counts, mutation scores, TSan results, timings, the arm64 run and *every hardware result in §8.2 and §8.13* are read from commit messages, design documents and committed bench logs, and are attributed as such. The 2 007 figure is a recorded measurement at `3ab1305`, not one taken here.
- **★★ Every boot in this document.** §8.2, §8.12 and §8.8 rest on runs this paper did not perform and could not perform — the box has no GPU. What was checked here is that each claim names a run, a revision, a machine and a committed log, and that the logs are present in `docs/reference/bench_evidence/` (87 files). **Their contents were sampled, not audited line by line.**
- **★ Whether the QEMU/KVM backend differential passes.** §8.14 reports that `crates/kayfabe-vmm-qemu/tests/kvm_different` exists and is the experiment the previous revision said only a second backend could run. **It was not executed here**, so whether the "zero trait changes" half of the agnosticism claim held is [unverified].
- **★ The current mutation score, deliberately and by the project's own rule.** The most recent campaign to complete is void (§7.7, Lie 5) and the one before it ran against a tree modified underneath it. A bar of 91 is in force; **no clean campaign has completed under it**. There is consequently no defensible score to state, and this document states none.
- **Anything about NVIDIA's 580 driver that this paper did not read itself.** The §8.7 claims about 580-vs-610 are taken from `docs/design/gsp_580_correction_brief.md` and `mode2_gsp_port_plan.md` §14, which state their own limit: only 580.159.04 and 610.43.02 are vendored and were read directly. Seven further driver tags used to narrow the element-layout break interval are **relayed from another agent's probe and were read by nobody in this chain** [unverified]. The predicate the project derived (`major >= 610`) is safe under either reading because the 610 boundary is the directly-verified one; the sentence "580, 590 and 595 are all on the 48-byte side" is not.
- **Whether the 580 resume path is ever taken.** §11-O7a is open, it is the sharpest risk in §8.7, and the evidence that would settle it — the contents of a sequencer buffer — is in closed GSP-RM firmware. Nothing here narrowed it.
- **"TSan, 0 races."** Not re-run; only the target list and the test count in those targets were checked.
- **The relationship between the C's 2026-06-16 bare-metal result and the 2026-07-25 one-process-per-lifetime finding.** The memory note flags this itself: the finding was not cross-checked against the bare-metal box, and warns against assuming the C regressed.
- **Whether Mode 2 ever produced any display output.** No positive evidence was found, only the plan. Absence of evidence.
- **★ Anything about the demand ladder or the CPU-RM recorder beyond its own documents** (§8.3). The classification is a static reading of the driver source, stated as one by its authors; the two numbers this paper prints were re-derived from the committed TSV, but the *arguments* behind each row were not re-checked. And ⊘ **the recorder's observational neutrality is not proven by anybody** — its author says so.
- **★★ The entire multi-tenancy confidentiality analysis** (§8.13). Every claim in it is [inferred] from driver source; both source documents open by saying no GPU was switched on for them; and the experiment that would settle the central finding — two VMs on one box — has never been attempted by anyone.
- **Whether `memmgrTestCeUtils` is really the next wall.** It is [inferred] from source and **no boot has ever reached it**. The escape-hatch analysis that makes it un-fakeable is a reading, and this paper checked that the readings are cited, not that they are right.

A Measurement method

Every number in this paper that is not attributed to a document was produced by one of the following, run against the kayfabe tree at `f33e1c8` and `/workspace/nvidia-gpu-passthrough` at `8001cb5`. **No compilation, no test execution and no network access was involved**, and on this box none was possible: it has no GPU and roughly 10 GB of free disk, so a `cargo build` was ruled out before writing began rather than skipped afterwards. The measurements were taken from a clean checkout of the pinned commit.

Quantity	Command
implementation lines	`find crates -path '*/src/*' -name '*.rs' | xargs wc -l`
test lines	the same over `tests`, `crates/*/tests`, `fuzz`
per-crate lines	the same, per `crates/<name>`
`#[test]` sites	`grep -rc '#[test]'` over workspace members, excluding the detached `crates/kayfabe-abi/gen` and the `trybuild` ui fixtures
`unsafe` relaxations	`grep -rhoE 'unsafe ({|fn |impl |trait )' crates/kayfabe-linux-raw/src/*_unsafe.rs | wc -l` — the same expression the CI ratchet uses
dependency edges	every `Cargo.toml` read in full, cross-checked against `Cargo.lock`
commit counts and dates	`git rev-list --count HEAD`, `git log --format=%ci`
document staleness	`git log -1 --format=%ci` per file, compared against the commit that invalidated the claim

The five figures are TikZ, drawn from the measurements above and from a function-by-function read of the call chains they depict; they are vector, not traced from an image. Figure 2 in particular is generated from the manifest edge list rather than from any design document, which is why it disagrees with the hexagon picture about which crates are "ports".

Two method rules carried forward, both prompted by the figures being where the drift was worst. Figure 3 names *symbols* rather than `file:line` pins, as the repository itself did to its own 105 pins in `3fd1455`. And Figure 2's edge list is re-derived from scratch every revision rather than edited — it was re-derived by script this time, which is how the `mocks` → `abi` edge and the new topological ordering were picked up without being looked for.

One method rule carried forward. Every claim carried over from the previous revision was re-measured rather than trusted, *including the ones this paper itself originated*. That is the only reason rows 27, 28, 30 and 33 exist — all four are claims this document asserted with no hedge, and all four had gone stale in the *understating* direction, which nobody audits for. **Re-derive, do not restate**, because a restated number and a measured number are indistinguishable on the page.

★ **One method rule added at this revision, and it earned two corrections immediately.** *The brief is a source like any other and is re-derived, not obeyed.* Two figures supplied to this revision as settled facts were checked against the tree and found wrong (§10 rows 31–32); one was a number that appears in no document, and one was a blanket where the exceptions carried the actual argument. Neither is a criticism of whoever wrote the brief — both were close to right, and the second was a fair summary of an earlier document that a later measurement had superseded. **The rule is that an instruction and a citation decay the same way.**

★ **The claim ledger was run against this tree, and the result is honest in both directions.** `scripts/claim_ledger.py --gate` returns **382 unattributed / 66 conflated / 17 bare-hardware**, all three bars holding, before and after every edit in this revision. ⊘ **And that is a weaker statement than it looks**: the ledger's universe is `*.rs` and `*.md`, so **this file is not in it**. The gate confirms the surrounding tree did not regress while the whitepaper was written; it says nothing whatever about the whitepaper. That gap is stated here rather than left for a reader to discover, because a green gate beside an unpoliced artifact is precisely the shape of *a green instrument on an unexercised path*.

One closing note on how to use this paper. If you are here to find what is wrong with kayfabe, the highest-yield reading order is: §8.2 (the execution plane cannot be faked, and it is the whole remaining problem), §8.8 (what contact with hardware did and did not buy — the conjunction, not either half), §3.1 (the oracle is positively wrong about a class of rows, and a larger class is unmeasured), §8.12 (an invariant broken on the production path under 1 975 green tests), §8.13 (there is no tenant axis, and the thesis is multi-tenancy), §7.7 (the evidence machinery has now been wrong five times, every one flattering), §8.6, §8.5 (an independent oracle that was independently written and identically wrong), and §8.7's open resume handoff, which still has no proposed resolution. Then the code — starting with `VerbPlan::gated_doorbell`, `Spine::apply`, `rmrpc::translate` and `kayfabe_fwd::plan_ce`. Do not start with `README.md`, and do not use `crate_maturity_map.md` for anything (§10 row 29).

And one note on how to read its confidence, which this revision sharpened. Each of the four revisions of this document has found its own predecessor wrong in several places, and **for three consecutive revisions the falsified claim was the one the predecessor led with**. This revision's lead claim is that the execution plane is the wall. It is the fourth answer to "what is the critical path", the previous three were each invalidated within days by the work they named, and *they were invalidated in the same direction every time*. Treat every unhedged sentence here the same

way — including this one, and including that one.